UNIVERSITY
OF OSLO

Master's Thesis

Model Evaluation with Mechanism-Isolated Time Series Simulations for Vector Borne Disease Forecasting

Ingar Benonisen
ingarabe@uio.no

Department of Informatics
Faculty of Mathematics and Natural Sciences

16th November 2025

Abstract

Vector-borne diseases (VBDs) are human illnesses transmitted by vectors carrying pathogens. Vectors, such as mosquitoes and parasites, are ectothermic and thrive in warmer climates. Climate change affects vectors' reproduction, behavior, and population dynamics, increasing their prevalence [6]. Consequently, the demand for predictive models, that enable healthcare systems to work proactively against VBDs, is also increasing [34].

Such predictive models require thorough evaluation before deployment, but data scarcity within climate–health often impedes this process [15]. Data simulation has been advocated for to supplement real-world data in model evaluation, as it enhances model transparency and enables hypothesis exploration by giving the user control over time series and their introduced signals [27]. However, simulation scripts are often ad hoc, model-specific, and difficult to reuse.

This study developed a time series simulation and model evaluation framework named `mestDS`. Employing the Design Science Research (DSR) methodology ensured research rigor and the development of a high-quality artifact. During the design process, a domain-specific language (DSL) proved suitable for defining simulations and model evaluations efficiently and flexibly, overcoming the cumbersomeness and *ad-hoc-ness* limitations of simulation scripts.

Furthermore, this study explored a relatively unexplored approach to evaluating VBD forecasting models through time series simulations. The core idea is to evaluate models based on their ability to capture mechanisms of the target's data generating process (DGP) in isolation. For example, a known mechanism in VBDs is the lagged relationship between rainfall and disease cases. By employing this approach, simulations introduce only this mechanism into the time series. Models should perform well on such simplistic time series before being applied to complex, real-world time series. For literary purposes, this approach is termed *Mechanism-Isolated Model Evaluations with Simulated Time Series (MIMES)* in this thesis, however, it should not be mistaken as a novel concept introduced by the author.

The latter part of the study combined the evaluation of `mestDS` with the exploration of *MIMES* on VBD forecasting models. The experimental results demonstrated that `mestDS` provided an efficient and flexible pipeline from simulation to model evaluation, largely due to the implemented DSL. Additionally, the findings demonstrated the value of the *MIMES* approach. Two mechanisms from the VBD DGP were evaluated on three different models, revealing varying performances in capturing seasonal patterns and the lagged relationship between rainfall and disease cases. These results provide indications into performances on real-world data and their learning requirements for specific mechanisms.

Acknowledgments

This thesis has not only provided me with valuable experience and knowledge in data simulation and machine learning (ML), but also showed me that determination, consistency, and hard work can make even the most challenging and tedious tasks, such as a Master's thesis, possible. I have the utmost respect for everyone who has completed such a journey.

I would like to thank my supervisor Geir Kjetil Ferkinstad Sandve, co-supervisor Knut Dagestad Rand, and researcher Chakravarthi Kanduri for their guidance, support, and in sharing their knowledge.

Lastly, I would like to thank my co-student Martin Hansen Bolle, for our collaboration on the development of `mestDS` and for the time we have shared together over the past two years.

Declaration of the use of generative artificial intelligence

In this scientific work, generative artificial intelligence (AI) has been used. All data and personal information have been processed in accordance with the University of Oslo's regulations, and I, as the author of the document, take full responsibility for its content, claims, and references. An overview of the use of generative AI is provided below.

- Generative AI has provided suggestions, guidance and debugging support during the development of the Python framework introduced in this thesis.
- Generative AI was used during literature reviews to discuss the content in question. It enhanced my understanding of certain topics, and impeded my understanding of other topics.
- Generative AI was prompted to act as a literary mentor, providing guidance and suggestions to improve readability and clarity, rather than simply supplying revised versions of the text. It was also used to correct grammatical errors.

Contents

List of Figures

List of Tables

List of Abbreviations

ML	Machine Learning
VBD	Vector-Borne Disease
DSL	Domain-Specific Language
DGP	Data Generating Process
DSR	Design Science Research
MIMES	Model Evaluation with Simulated Time Series
GPL	General Purpose Language
RMSE	Root Mean Square Error

Chapter 1

Introduction

The global environment is experiencing rising temperatures and more frequent and intense extreme weather events at devastating costs. Among the implications these changes have caused and continuing to cause, is the rising prevalence of vector borne diseases (VBD). VBDs are human illnesses transmitted by vectors carrying pathogens and cause over 700,000 deaths annually [31]. Arthropod vectors, such as mosquitoes and ticks, are ectothermic and strives in warmer climates. Hence, their reproduction, behavior, and population dynamics are affected by climate change [9].

To mitigate these changes, forecasting disease outbreaks with ML models has been advocated for [9]. With knowledge of when an outbreak will occur, the health care systems are able to proactively mitigate the burden of VBD. The deployment of such models requires rigorous assessment, however, the scarce amount of available data impede this [15].

Data simulation offers a solution to the scarce data amount. In addition, data simulation enables transparency of the models, and evaluating different hypothesis, by giving the user control of the datasets and their characteristics [27]. On the other side, there is a lack of generic and flexible frameworks to support this. Simulation scripts are often ad hoc, model specific, and difficult to reuse.

1.1 Objectives

The objective of this thesis is to explore the development and applicability of a data simulation framework that solves the cumbersomeness of data simulation. The aim is to develop a generic framework that allows the user to simply and effectively simulate multiple datasets with different desired characteristics, and subsequently allow users to effectively evaluate a model in a transparent manner. The framework should facilitate the entire pipeline, from data simulation to model evaluation.

Lastly, the thesis aims to explore the possibilities of using simulated time series for evaluations of disease forecasting models.

1.2 Contributions

The thesis contributes to the field of disease forecasting with a framework that allow researchers and developers to efficiently evaluate models and improve model re-development. The experiments with using simulated time series in evaluating disease forecasting models gives insights into how the framework can be utilized and how data simulation improves model evaluation and development in general.

1.3 Structure

The thesis begins by presenting relevant concepts, theories, and previous research in the Background chapter. It then proceeds to the Research Methodology chapter, which introduces Design Science Research and how it has been employed in the thesis, to ensure research rigor and a quality artifact. Continuing, the chapter Framework Design describes the final framework in detail, whilst the chapter Implementation describes the development process. Furthermore, Results and Experiments demonstrates how the framework can be utilized for its intended purpose. Finally, the thesis concludes with the Discussion and Conclusion chapters.

Chapter 2

Background

2.1 Climate Change

Climate change poses a fundamental threat to human health and the systems that sustain it. Anthropogenic activities have led to increased green house gas emissions, resulting in higher temperatures and environmental shifts. These changes are causing more frequent and intense extreme weather events, such as heat waves, floods, and droughts [3].

Recent data underscores that climate change is undeniable. The past ten years (2015–2024) have been the warmest on record. In 2024, global temperatures reached 1.55 ° C above preindustrial average, making it the hottest year ever recorded [33]. This surpassed the previous record set just a year earlier in 2023 [35]. Reports from National Oceanic and Atmospheric Administration show an increase in extreme weather events in the United States. The 1980-2024 annual average is 9 events, whilst the 2020-2024 annual average is 23 events [30].

The increasing prevalence of extreme weather events has direct and indirect implications for human health. For instance, heatwaves can exacerbate cardiovascular diseases, or lead to the development of it. Similarly, air pollution can worsen or lead to respiratory illnesses. Floods and heavy rainfall can compromise water quality, facilitating the spread of waterborne diseases. Droughts and floods can devastate crops, leading to food shortages, which has severe consequences. Climate change is also known to cause psychological issues. Stress, anxiety, post-traumatic stress disorder, etc. can be caused by extreme weather events or other consequences of climate change, such as migration and economic instability. Research also documents the prevalence of "eco-anxiety", a distressed reaction to the current and future environmental state and losses [1].

2.2 Vector-borne Disease

VBDs are human illnesses caused by parasites, viruses, or bacteria that are transmitted by vectors (organisms such as mosquitoes or ticks) that carry and spread pathogens between humans, or from animals to humans. Vectors ingest disease-producing microorganisms during a blood meal and later transmit them to a new host after the pathogen has replicated. Once infectious, many vectors can transmit pathogens throughout their lifetime.

VBDs account for more than 17% of all infectious diseases and are responsible for more than 700,000 deaths each year, and include diseases such as malaria, dengue, chikungunya, yellow fever, Zika virus, West Nile fever, leishmaniasis, Chagas disease, and Japanese encephalitis . Malaria, a parasitic infection transmitted by Anopheles

mosquitoes, causes around 249 million cases globally and leads to over 608,000 deaths annually, with most fatalities occurring in children under five years old. Dengue is the most prevalent vector-borne disease, placing over 3.9 billion people in more than 130 countries at risk, and causing an estimated 96 million symptomatic cases and around 40,000 deaths each year.

The burden of these diseases is disproportionately high in tropical and subtropical regions, particularly among the poorest and least developed communities. VBDs not only cause illness and death but also perpetuate poverty through missed workdays, healthcare expenses, and long-term disability. Some diseases, such as leishmaniasis and lymphatic filariasis, can result in chronic suffering and stigmatization.

A variety of factors drive the distribution and spread of vector-borne diseases, including demographic changes, urbanization, global travel and trade, and climate change. Outbreaks in recent years, such as those of dengue, malaria, chikungunya, yellow fever, and Zika, have increasingly overwhelmed health systems in affected regions [31].

Climate change plays a significant role in the dynamics of VBDs. As arthropods are ectothermic, their development, survival, and reproductive rates are heavily influenced by temperature. For example, temperatures between 28°C and 30°C can enhance the development of Aedes aegypti mosquitoes and shorten the pathogen incubation period within the vector, thereby increasing transmission potential [6]. Warming temperatures have expanded the geographic and seasonal range of many vectors, allowing them to establish in previously unsuitable regions, such as parts of southern Europe [25].

In addition to temperature, changes in precipitation also affect vector populations. Increased rainfall and flooding can create new breeding habitats for mosquitoes, such as stagnant water in natural or artificial containers like ponds, irrigation channels, drainage ditches, and even water storage containers [14]. Conversely, drought conditions may cause communities to store water in ways that create additional breeding sites, or cause rivers to slow and form stagnant pools that become habitats for larvae [6, 9].

Preventing and controlling vector-borne diseases requires a comprehensive and coordinated effort at multiple levels. Central to this is strengthening vector control programmes through improved technical capacity and infrastructure, as well as enhanced monitoring and surveillance systems. These efforts help countries to better detect outbreaks early and respond effectively to reduce the number of cases.

Community engagement and public awareness are also crucial interventions. Changing behaviors and educating people about how to protect themselves and reduce vector habitats can greatly decrease disease cases. For example, simple actions like managing water storage or eliminating stagnant water sources in and around homes can reduce mosquito breeding sites. WHO supports these efforts by working closely with governments and partners to improve public knowledge and encourage active participation [34].

In addition, improving access to water and sanitation infrastructure is an important part of long-term disease control. Poor water management can create environments favorable for vectors, so efforts to enhance sanitation help reduce vector populations at the community level. By combining technical support with social mobilisation and improvements in environmental conditions, countries can make vector control more effective and sustainable. These strategies, supported by WHO's global guidance and resources, contribute significantly to reducing the burden of vector-borne diseases worldwide [34].

In summary, vector-borne diseases pose a substantial and growing threat to global

health, especially in vulnerable regions. Their control and prevention require a comprehensive and adaptive approach that includes vector surveillance, public education, infrastructure improvement, and international cooperation [31].

2.3 Disease forecasting

Part of the preventive actions to mitigate the burden of VBDs, are monitoring and prediction of VBD cases. VBDs' complex and dynamic nature complicates employing interventions for outbreaks, such as allocating limited resources or disseminating knowledge, in a timely and efficient manner. By predicting VBD cases, the preventive actions can proactively, opposed to reactivate, be employed where outbreaks are predicted to occur [34].

A recent systematic review of dengue outbreak prediction models provides valuable insight into current modeling practices, their limitations, and areas for improvement [18]. This review evaluated 99 models across 64 studies, encompassing a wide range of models and their inherent form, performance and documentation. Among the models, 60% employed statistical approaches, such as linear, Poisson, autoregressive models, and generalized additive models. The remaining 40% utilized ML methods, including random forest, support vector algorithms, neural networks, gradient boosting, and long short-term memory models.

Despite the growing use of ML and advanced time series techniques, the review highlighted three key inadequacies in current modeling practice: (1) A heavy reliance on secondary data, such as government reports, rather than real-time or actively collected data. (2) Limited inclusion of non-climatic features, impeding to capture the holistic dengue transmission dynamics. (3) Inadequate reporting of methodology, validation processes, and model performance metrics.

Regarding features, 70% of the models relied solely on climatic variables, primarily temperature, rainfall, and humidity. A smaller subset incorporated additional features such as demographic, entomological, and climate change-related factors, which improved model comprehensiveness. For example, entomological indicators like Aedes aegypti index or ovitrap egg counts, and demographic data such as population density and access to piped water, were used in some models. Models that combined climatic and non-climatic features tended to perform better.

The review also emphasized the importance of validation in model development. While 76% of models reported internal validation, only 5% included both internal and external validation, and the remaining 20% did not report any validation at all. Internal validation assesses a model's performance on unseen data from the same source as the training data. On the other hand, external validation evaluates the model on independent datasets, from different geographic regions, time periods, or populations, to assess the models performance under different circumstances or when validated by others[29]. This is especially important for model redevelopment, where predictive performance can be improved through updating the model based on insights from the external validation, such as adding or removing features [22].

Several studies in the review failed to account for time lag effects, despite the well-documented delay between climate conditions and vector response. This can substantially impair prediction accuracy.

Another study also highlights key areas for improvement in dengue forecasting models [15]. Johansson et al. argue that, despite a substantial body of research on dengue epidemic forecasting, very few models are actually applied in public health practice. This

is due to several limitations: forecasts fail to address the needs of public health services, do not provide probabilistic forecasts, lack external validation, and have insufficient forecast skill.

Furthermore, the study evaluates the performance of several models developed as part of the study. The results indicate that seasonal cycles are generally predictable, and in some cases, outbreaks can be predicted several months ahead. However, high-incidence epidemics remain difficult to predict accurately.

Lastly, the study emphasizes that the lack of publicly available datasets hinders the assessment of a model's generalizability. The authors argue that more open datasets are needed to properly evaluate both the forecast skill and the external validity of predictive models.

2.4 Time Series Data Simulation

Data simulation is the process of generating synthetic datasets that mimic real-world characteristics. Generating datasets through software, opposed to collecting data in the real-world, enables unconstrained size and user control over introduced signals [27]. This section delves into the characteristics of time series and the benefits and disadvantages of simulating time series.

2.4.1 Time Series

A time series is a sequence of data points collected or recorded at successive time intervals, typically equally spaced. Examples include the price of a stock, daily temperature measurements, or the number of dengue cases reported each month.

Time series data often includes underlying patterns that can be decomposed into three main components: trend, seasonality, and residuals. The trend captures the long-term progression or direction of the data, independent of short-term fluctuations. For example, a stock that increases in value over several years shows a positive trend.

Seasonality refers to patterns that repeat at regular intervals due to seasonal or cyclical effects. An example is the number of visitors to a beach, which tends to increase in summer and decrease in winter.

Residuals, also known as the irregular or random component, are the variations in the time series that remain after both trend and seasonality have been removed. These fluctuations are less predictable and can result from sudden external events. For instance, a sharp increase in dengue cases due to a local outbreak, which does not follow the usual trend or seasonality, would appear in the residuals [16].

To illustrate these components, consider a time series of temperature measurements over several decades. Due to climate change, there is a positive trend, with average temperatures increasing over time. Seasonal patterns occur within each year as temperatures rise in the summer and fall in the winter. Finally, short periods of extreme temperatures, such as heat- or cold waves represent residuals, as they are irregular and not explained by the trend or seasonal cycle.

2.4.2 Data Generating Process (DGP)

Each point in a time series is based on some sort of mechanism. For instance, the price of a stock is based on multiple factors: supply and demand; company-centric factors, such as sales and public relations; inflation and economic growth; and white noise. This mechanism is referred to as the DGP, and can be separated into deterministic

and stochastic, as time series naturally are either one of those. A deterministic process describes a time series that is predictable over time, such as the motion of a pendulum. On the other side, a stochastic process describes a time series with inherent randomness, such as the price of a stock.

In an ideal world, we would have full knowledge of all the factors and mechanisms driving the time series, allowing us to define the true DGP in exact mathematical terms. This would lead to perfectly accurate forecasts. However, in practice, such complete understanding is rarely possible. Instead, the goal becomes to approximate the DGP as closely as we can using a statistical or ML model. The better this approximation reflects the underlying reality, the more useful and accurate the resulting forecasts or analyses will be [16].

Having some understanding of the DGP is important when developing models, evaluating them, and simulating data. For example, when building a model to predict cases of vector-borne diseases, it is useful to know that there is often a time delay between heavy rainfall and an outbreak. This kind of information should be considered either when choosing a model or during the data processing step (e.g. adding lagged features). Also, when evaluating a model, it is important to keep in mind that sudden and random outbreaks can happen. A good evaluation should check how the model performs in such situations. Finally, when simulating time series data, knowledge of the DGP helps to create datasets that make sense and reflect realistic patterns.

2.4.3 Data Simulation

Simulating time series under controlled conditions enables the exploration of rare or extreme events, and the generation of time series where real-world data are scarce, missing, or too costly to collect.

There are multiple approaches to generating time series data. One approach, mechanistic (or process-based) simulation, relies on predefined rules or algorithms that define the underlying processes of the DGP. Another approach is parametric simulation, where statistical models such as ARIMA [28] are specified with chosen parameters to generate time series.

While the previous approaches do not strictly require existing data, there are also methods that generate time series based on real-world data. For instance, block bootstrapping is a resampling method that maintains temporal structure [2]. Deep generative models are also utilized in time series generation. Neural networks such as GANs (e.g., TimeGAN) [36] and VAEs [17] can be trained to produce time series that capture complex temporal dependencies. However, these approaches can inherit the limitations of real-world data [8].

Each approach serves a distinct purpose, balancing realism, interpretability, and complexity. This thesis focuses on the mechanistic simulation approach, which offers the flexibility to generate controlled, interpretable, and biologically or physically meaningful time series for rigorous model evaluations.

2.4.4 Motivation for Simulating Time Series Data

Although real-world time series are mostly used to train and evaluate models, they come with significant limitations.

Data quality is a primary concern. Time series can include noisy or adversarial signals, which can impair model robustness, and potentially lead to overfitting.

Additionally, models trained on real data may inherit existing biases and vulnerabilities, limiting their generalization [8].

Another key issue is the difficulty of ensuring good generalization. A model that performs well on one dataset may fail on others due to distributional shifts, sudden changes, or unseen events [8]. This is particularly critical in time series domains, where the future is not always like the past.

Furthermore, many domains face a fundamental problem: data scarcity. In these fields, real-world data may lack size, resolution, or sufficient controls, making model evaluation uncertain and sensitive to choices like hyperparameters. As a result, reported model performance may be due to fine-tuning, opposed to real advances [27].

Data simulation offers a complement to the limitations of real world data. First, time series can be simulated at unconstrained size and quantities. Second, simulating time series gives the user full control over introduced signals and data assumptions. This allows researchers to evaluate models under rare and extreme cases, which potentially are not available in real-world datasets. Furthermore, it allows researchers to assess models' robustness to domain shift, and enables reproducible and transparent evaluations. By simulating data that reflects the DGP, researchers can make more rigorous evaluations of their models. Ultimately, simulation should be seen as a complement, not a replacement, to real-world data [27].

2.5 Model Evaluation

> "Think of training a model like teaching a student. Model evaluation is like giving them a test to see if they truly learned the subject—or just memorized answers." — GeeksForGeeks [19].

The GeeksForGeeks analogy captures the essence of model evaluation: a model is trained and then tested on unseen data to estimate its predictive performance. However, this thesis distinguishes between *performance estimation* and *model evaluation*, arguing that evaluation should go beyond simply measuring predictive performance on a held-out dataset. While performance estimation focuses on producing a single performance score, model evaluation should also include a deeper analysis of the model's internal behavior and its performance under varying conditions [26].

2.5.1 Performance Estimation

Solving a problem with ML typically involves multiple iterations of performance estimation. The workflow from identifying a problem to deploying an ML solution includes, among other steps, selecting an appropriate model, tuning its hyperparameters, and determining whether the problem is solvable with ML at all. Different problems require different model architectures, and selecting the best one involves comparing performance estimates. Furthermore, optimal performance often requires careful hyperparameter tuning, which involves comparing performance with different hyperparameter values. Finally, once the best-performing, fine-tuned model is identified, its performance estimate provides an indication of whether it is suitable for the target application [23].

There are several performance estimation methods and metrics, suited for different applications. Naturally, different ML tasks, such as classification and regression, requires different metrics. Furthermore, the application of one regression model might require a different method and metrics than another application of a regression model. To

illustrate this, consider an example (not drawn for the literature, but rather made up for demonstrative purposes): one model is developed to predict house prices and another model is developed to predict VBD cases. The former application does not include a temporal dependency, as each observation in a dataset is independent. On the other side, the latter application introduces temporal dependency across the observations. This requires a careful selection of performance estimation method. While the former application can implement cross validation, a common resampling method, the latter model must implement the temporal dependency-friendly alternative time series cross validation.

Furthermore, for the purpose of this example, let us assume that the former and latter applications are respectively homo- and heteroscedastic. This implies that a constant error variance is expected for the former. For instance, the residual error is approximately constant for houses worth $500,000 and $1,000,000. In contrast, the latter application is expected to have a varying error variance over time, meaning that forecast uncertainty may increase during certain periods, such as during specific seasons or disease outbreaks. Similarly to selecting performance estimation method, the metrics chosen for both applications require careful consideration. Root Mean Square Error (RMSE) might be suitable for the former application, as it punishes large errors. However, RMSE might not be as suitable for the latter application, since it squares the error, causing high-variance periods to dominate the final score, resulting in an biased performance estimation. In contrast, Mean Absolute Percentage Error, where the error is relative to the true value, might be a more suitable metric in such heteroscadisitc cases.

2.5.2 Model Evaluation

As previously discussed, this thesis distinguishes between performance estimation and model evaluation. While in some domains, for instance image classification, a single performance estimation on a sufficiently representative dataset may provide enough insight, this is rarely the case for time series forecasting [26]. In image classification, representativeness might mean including images with varying lighting, orientations, and object characteristics. In contrast, for VBD forecasting, a single dataset may fail to capture the various mechanisms from the DGP.

For instance, both drought and heavy rainfall can, after a period of time, increase the number of VBD cases [6, 14]. A robust evaluation should test a model's ability to predict under both circumstances. VBDs also exhibit seasonal patterns and sudden outbreaks; evaluation should therefore consider both. These scenarios are not necessarily present in a single dataset.

Furthermore, evaluation should assess a model's learning requirements. Scarce, irregular data can reduce model accuracy and generalization. An effective evaluation should therefore determine the minimum data requirements, both dataset length and frequency of events, needed for the model to learn relationships, such as the lagged correlation between rainfall and outbreaks.

In summary, while performance estimation involves training and testing a model using specific methods and metrics, model evaluation should go beyond assessing performance on a single dataset. It should also assess key aspects of the domain's DGP, performance on extreme events, the model's learning requirements, and its robustness and interpretability. Such model evaluation provides a more comprehensive understanding of a model's performance and capabilities and enhances trustworthiness in real-world deployment.

2.6 Mechanism-Isolated Model Evaluation with Simulated Time Series (MIMES)

So far in this thesis, it has been argued that model evaluation in VBD forecasting is constrained by data scarcity and available public datasets, impeding model evaluation [15], and as a result reported performances may be due to fine-tuning, opposed to real advances [27]. Furthermore, the thesis has pointed out several advantages for using simulated time series as a complement to real-world time series in model evaluations. First, simulated time series address weaknesses of real-world time series, such as noise, adversarial signals, biases or distributional shifts [27, 8]. Second, simulated time series gives the researcher the control over the introduced signals, which enables the exploration of hypothesis and transparency in model evaluations [27]. Moreover, the thesis argues that model evaluations should go beyond simply estimating model performance on a single dataset, but rather consider the broad range of introduced mechanisms of a DGP.

Building on this and Sandve et.al.'s argument that models should perform well on simplistic simulations before being evaluated on real-world data [27], an extension to model evaluation is to evaluate models on simplistic, mechanism-isolated time series. More precisely, this approach implies that a single known mechanism within a DGP is simulated and evaluated on a model. Such evaluation enables researcher to test if a model is able to capture a known, core mechanism, without interference from other mechanisms or noise. In doing so, it provides an additional validation that examines if a model's correct predictions on real-world data arise from accurately capturing the underlying mechanism or not.

To refer to this approach more concretely, this thesis use the term *Mechanism-Isolated Model Evaluation with Simulated Time Series (MIMES)*. The term does not represent a new, novel idea introduced in this thesis, but rather as conceptual label for a unexplored approach to model evaluation. *MIMES* refers to the process of evaluating models on simulated time series that represent mechanisms of a DGP in isolation, and reflects the idea that models should perform well on simplistic simulated time series, before moving being evaluated on real-world time series.

In the context of VBD forecasting, adopting *MIMES* for model evaluation could improve robustness, a limitation pointed out by previous research [15, 18]. In practice, this involves simulating known VBD mechanisms, as discussed in section 2.2. For example, using *MIMES*, the lagged relationship between rainfall and disease cases could be systematically explored, a mechanism which models have struggled to capture [18]. By doing so, *MIMES* would complement traditional model evaluation, providing a more holistic picture of model performance and enhancing model interoperability.

2.7 Domain Specific Language (DSL)

Fowler defines DSL as "a computer programming language of limited expressiveness focused on a particular domain" [7]. When developing a DSL, a underlying framework which parses the DSL and executes code naturally occurs. Furthermore, Fowler's definition has four key elements:

- **Computer programming language:** A DSL is used by humans to instruct a computer, meaning it must be executable like any other programming language, yet easy to understand.

- **Language nature:** As a language, a DSL should support fluent and composable syntax, making it expressive and easy to read for its intended domain.
- **Limited Expressiveness:** Unlike general-purpose languages, a DSL avoids broad abstraction mechanisms and instead includes only the minimum necessary features for the domain it serves.
- **Domain focus:** Its design is centered for a specific domain, which justifies its limitations in expressiveness and scope.

These characteristics distinguish DSLs from general-purpose programming languages and enable them to act as concise and readable interfaces over underlying frameworks [7].

Moreover, Fowler separates DSLs into external, internal and language workbench. External DSL is a language different from the general purpose language (GPL) used in the underlying framework. A script of an external DSL is parsed by the framework. In this case, the DSL can be viewed as a thin, readable layer over the underlying framework. An external DSL could have its own syntax, or utilize existing languages such as XML or YAML.

On the other hand, an internal DSL uses the same GPL as the underlying framework. In this case, the abstraction of a "thin layer" and the differences between a DSL and a API, becomes more vague. However, an essential part of internal DSLs is that is should feel more like a custom language.

Without going into detail about the last category, language workbenches are essentially IDEs for building and defining DSL.

In practice, DSLs offer several advantages. First, DSLs improve developer productivity by allowing domain-specific logic to be expressed more clearly and concisely, reducing the likelihood of bugs and making code easier to maintain. Second, they enhance communication with domain experts by offering a syntax they can often read and validate, even if they do not write the code themselves. This enables a shared understanding and collaboration. In addition, DSLs support abstraction by decoupling domain logic from complex implementation [7].

2.8 HISP/DHIS2 and chap-core

This thesis project stems from the HISP Centre at the University of Oslo's (UiO) research group *Information Systems* [13]. HISP was initially a collaborative research project between the UiO and the University of Western Cape to support digitization of decentralized health system management in post-Apartheid South Africa, which resulted in *District Health Information Software* (DHIS) [11]. Today, HISP is a global network of information system experts, across the world, engaging in more than 80 countries, supporting digitalization and data use [12]. Furthermore, the initial DHIS has evolved into a second version, DHIS2, implemented in over 100 countries [4].

More specifically, the thesis project stems from the researchers working on DHIS2's Chap & Modeling Platform, which unifies climate health models under one ecosystem, facilitates modeling workflows and model evaluations. Part of the platform is `chap-core`, which functions as the backend system, as well as a standalone Python package and command line tool [5].

A key objective of mestDS, is to integrate with `chap-core`, enabling researchers and developers to combine the use of `mestDS` and `chap-core`, to evaluate models using

simulated time series. Models integrated with `chap-core` follow a `MLflow` architecture [21] and a data structure discussed in 4.4.1. Such models are referred to as `chap-models` in this thesis.

Chapter 3

Research Methodology

This chapter outlines the methodological approach adopted in this thesis, which is based on the principles of DSR. Given the thesis's objective to develop a practical and theoretically grounded framework for evaluating forecasting models through data simulation, DSR offers an appropriate paradigm. It emphasizes the development and evaluation of artifacts that address real-world problems while cxontributing to academic knowledge.

The chapter begins by introducing the DSR paradigm and it's key characteristics. This is followed by a detailed mapping of how DSR was applied throughout the project, including how the artifact was developed and evaluated in accordance with DSR principles. By mapping the research process to the guidelines, the chapter demonstrates how methodological rigor and relevance were achieved through an iterative, artifact-centered approach.

It is important to emphasize that the DSR guidelines were not followed strictly, but rather served as a conceptual compass. The design process unfolded organically, with the principles of DSR emerging naturally as part of the iterative development and reflection throughout the thesis.

3.1 Design Science Research

DSR is a research paradigm that aims to create artifacts, such as models, methods, constructs, or instantiations, that solve relevant, real-world problems. Its roots lie in engineering disciplines, where the primary focus is on creating artifacts that contribute to the practical application of knowledge. DSR is distinguished by its focus on both theoretical contribution (e.g. advancing knowledge) and practical application (e.g. solving specific problems in a real-world context).

In DSR, researchers address important problems by designing artifacts that are rigorously evaluated to assess their effectiveness and utility. The evaluation of these artifacts is critical for demonstrating their practical value and ensuring that they meet the needs of stakeholders. Additionally, DSR emphasizes an iterative process, where designs are refined and improved through continuous cycles of testing, feedback, and modification.

Table 3.1 lists the guidelines for DSR, suggested by Hevner et al. [10], which help ensure that DSR produces high-quality, relevant artifacts:

Table 3.1: Guidelines for Design Science Research, adapted from Hevner et al. (2004) [10].

Guideline	Description
1. Design as an Artifact	Design-science research must produce a viable artifact in the form of a construct, a model, a method, or an instantiation.
2. Problem Relevance	The objective of design-science research is to develop technology-based solutions to important and relevant business problems.
3. Design Evaluation	The utility, quality, and efficacy of a design artifact must be rigorously demonstrated via well-executed evaluation methods.
4. Research Contribution	Effective design-science research must provide clear and verifiable contributions in the areas of the design artifact, design foundations, and/or design methodologies.
5. Research Rigor	Design-science research relies upon the application of rigorous methods in both the construction and evaluation of the design artifact.
6. Design as a Search Process	The search for an effective artifact requires utilizing available means to reach desired ends while satisfying laws in the problem environment.
7. Communication of Research	Design-science research must be presented effectively to both technology-oriented and management-oriented audiences.

3.2 Application of DSR Guidelines

Design as an Artifact & Problem Relevance

As discussed in chapter 2, data simulation and DSL are viable approaches for addressing the challenges of model assessment of VBD forecasting models. These concepts serve as the theoretical foundation for the artifact developed in this study. The artifact's design, which builds upon these foundations, is discussed in further detail in chapter 4.

Design Evaluation, Research Rigor & Design as a Search Process

As previously mentioned, DSR emphasizes on an iterative design process. Throughout the project, an iterative development cycle was employed, where evaluations of the artifact, feedback from the supervisor and continuous literature review have been used to improve the design artifact. The iterative approach has allowed for great flexibility, which has been beneficial in a project where the end result was unknown. During the iterative development of the artifact, multiple paths have been pursued, in search of improving the artifact's design.

The development started with three simplistic pilot projects defined by the supervisor, each incrementally increasing the complexity of the simulation. These pilot project was initiated at the beginning of the thesis, prior to any literature review, to provide a practical introduction to data simulation and establish a foundation for further

development. Following the pilot projects, further work was done to develop upon the foundation.

Also, during final stage of development, the artifact was evaluated on different test cases, such as simulating datasets with certain characteristics and evaluating different models. This additional evaluation uncovered framework weaknesses, which were subsequently addressed. As a result, the design artifact's utility, quality, and efficacy were significantly improved.

Research Contribution

The thesis aims at contributing with an artifact that solves the problems with VBD forecasting model assessment discussed in chapter 2. Furthermore, the thesis aims at contributing with knowledge within this topic, e.g. whether DSL truly is a viable option for data simulation and model assessment or refrain from pursuing non-viable approaches tested during the development process. Lastly, the thesis also contributes with insights from experiments that employ the *MIMES* approach to model evaluations.

Communication of Research

In order to effectively present the artifact, a user guide has been created for several iterations during the design process. The user guide gives insights into how the artifact can be used to solve the cumbersome problem with VBD forecasting model assessment and the life cycle of the framework. Furthermore, several models was evaluated on a number of datasets. Both the result (generated model evaluation report) and the set up (DSL) from the model evaluations also gives insight into how the framework can be utilized.

Chapter 4

Framework Design

This chapter presents the final design of the framework developed to support the simulation of climate-health data and the evaluation of models for VBD forecasting. Grounded in the principles of DSR, the framework has been iteratively developed to ensure flexibility, modularity, and practical relevance.

The source code is publicly available on *GitHub*, and the package can be installed from *Python Package Index (PyPi)*. An installation guide and framework tutorial are provided in a complementary *GitHub* repository. Links are listed in Appendix A.

The following sections provide a detailed overview of the architecture, the individual components and their interactions, and the end-to-end pipeline, from defining simulation and evaluation configurations in the DSL to generating model evaluation reports.

4.1 Architecture Overview

The design centers around a modular architecture composed of a DSL and three core components: `mestDS`, `Simulator`, and `Evaluator`. The DSL enables concise, high-level definitions of simulation and evaluation configurations. The `Simulator` and `Evaluator` components respectively handle data simulation and model evaluation, while `mestDS` parses the DSL and manages the instances of the other two components.

Figure 4.1 provides an overview of the framework and illustrates the relationship between its components. In a typical workflow, the DSL script is passed to the `mestDS` component, which parses it and generates instances of `Simulator` and `Evaluator`. The `mestDS` component exposes various functions, such as triggering simulations or initiating model evaluations through the corresponding component instances.

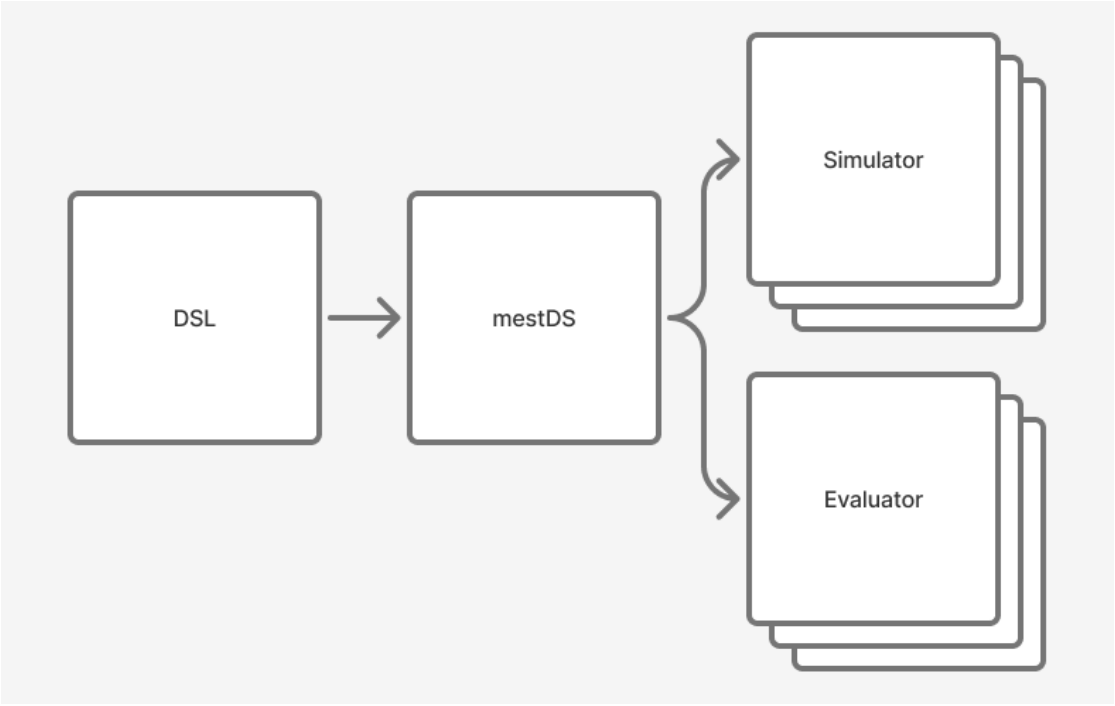

Figure 4.1: Framework overview

4.2 Domain-specific Language

The DSL developed for this framework has undergone several iterations, with each iteration aimed at improving simulation flexibility, reusability, and overall effectiveness.

The DSL consists of two main components: the configuration of simulators and evaluators. These configurations are effectively forms that populate the corresponding classes discussed in Section 4.6 and Section 4.8. The simulator configuration can itself be divided into two parts: the definition of public functions and lists (which are reusable across multiple simulators), and the definition of simulator instances.

This section presents selected snippets of a DSL script to illustrate the structure and design of the language. A complete example is provided in Appendix B.

4.2.1 Defining Simulators

Listing 4.1 shows how a `Simulator` instance is defined in the DSL. Each instance is included as a YAML mapping within a list assigned to the top-level key `simulators`. The DSL populates the required attributes of the `Simulator` class through this configuration.

In the example below, the simulator generates a synthetic dataset where the variables `population` and `mean_temperature` follow a normal distribution with fixed means and small noise. The variable `rainfall` is typically flat, with periodic seasonal spikes and one random spike in the test set. Finally, `disease_cases` are calculated based on the rainfall observed three months earlier.

```
simulators:
  - id: 1
    name: train with no random spike
    description: Is the model detecting the seasonality or time lag?
    time_delta: M
```

```
length: 100
x:
  - name: rainfall
    function_ref: get_rainfall_spike
    params:
      random_spikes: [93]
  - name: population
    function: |
        def get_flat_value():
            return np.random.normal(1000, 5)
  - name: mean_temperature
    function: |
        def get_flat_value():
            return np.random.normal(27, 3)
y:
  - name: disease_cases
    function_ref: get_disease_cases_with_lag
    params:
      lag: 3
```

Listing 4.1: Defining a Simulator instance in DSL

Public Functions and Lists

In Listing 4.1, the variables `population` and `mean_temperature` define their logic using inline Python functions via the `function` mapping key. In contrast, `rainfall` and `disease_cases` use the `function_ref` key to reference public functions defined elsewhere in the script. Additional arguments for each function are supplied using the `params` key.

To improve code reuse and maintainability, the DSL supports defining public functions and lists. These shared resources can be referenced by multiple simulator configurations. Listing 4.2 illustrates the definition of such public components. Notably, the functions used for `population` and `mean_temperature` are identical apart from their parameters in listing 4.1. Instead of duplicating the function, it is added to the public `functions` block and referenced using `function_ref`.

Similarly, the public list `seasonal_spikes`, defined under the `lists` key, is used as an input parameter to the function `get_rainfall_spike`.

```
public:
  functions:
    get_flat_value: |
      def get_flat_value(mean, randomness):
        return np.random.normal(mean, randomness)
    get_rainfall_spike: |
      def get_rainfall(i, seasonal_spikes, random_spikes):
        if i in seasonal_spikes or i in random_spikes:
          return 50
        else:
          return 10
    get_disease_cases_with_lag: |
      def get_disease_cases_with_lag(rainfall, lag):
```

```
        if len(rainfall) < lag:
          return int(rainfall[-1])
        return int(rainfall[-lag])
  lists:
    seasonal_spikes: [5, 17, 29, 41, 53, 65, 77, 89]
```

Listing 4.2: Public functions and lists in the DSL

Inheritance

In scenarios where a user wants to evaluate a model on datasets that differ only slightly, it may be inefficient to redefine entire simulator configurations. For instance, Listing 4.1 defines a simulator with seasonal rainfall spikes and a single random spike in the test set. Suppose the user wishes to evaluate the model on a similar dataset that includes an additional random spike in the training set.

Rather than duplicating the entire configuration, the DSL supports inheritance through the `inherit` key, which references the `id` of an existing simulator. This enables a new configuration to override only the necessary parts.

Listing 4.3 shows how this inheritance works in practice.

```
simulators:
  - id: 1
    name: train with no random spike
    description: Is the model detecting the seasonality or time lag? The m
    time_delta: M
    length: 100
    x:
      - name: rainfall
        function_ref: get_rainfall_spike
        params:
          random_spikes: [93]
      - name: population
        function_ref: get_flat_value
        params:
          mean: 1000
          randomness: 5
      - name: mean_temperature
        function_ref: get_flat_value
        params:
          mean: 27
          randomness: 3
    y:
      - name: disease_cases
        function_ref: get_disease_cases_with_lag
        params:
          lag: 3
  - id: 2
    inherit: 1
    name: train with one random spike
    description: In this case, the model is trained with one random spike a
    x:
```

```
  - name: rainfall
    params:
      random_spikes: [56, 93]
```

Listing 4.3: Defining a second Simulator instance

4.2.2 Defining Evaluators

The DSL also supports the configuration of `Evaluator` instances. This configuration is relatively straightforward, as the `Evaluator` component is less complex than the `Simulator`.

Listing 4.4 shows how two evaluators can be defined. Each evaluator specifies a reference to a model along with several evaluation parameters, such as prediction length, evaluation stride, number of test sets, metrics to compute, and plotting length.

The second `Evaluator` configuration references a model designed for weekly time series data. To accommodate this, the associated `Simulator`'s `time_delta` and `length` attributes are overridden via the `time_delta` and `sim_length` keys in the evaluator definition. Additionally, this configuration adjusts other parameters such as prediction length, stride, and the number of test sets. This illustrates the flexibility of the DSL, allowing customization to support varying model requirements.

```
evaluators:
  - model: https://github.com/dhis2-chap/ewars_template.git
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    metrics: [mse, pocid, theils_u]
    plot_length: 100
  - model: models/weekly_ar_model/
    sim_length: 300
    time_delta: W
    prediction_length: 12
    stride: 12
    n_test_sets: 3
    metrics: [mse, pocid, theils_u]
    plot_length: 100
```

Listing 4.4: Defining Evaluator instances in the DSL

4.3 mestDS

The `mestDS` component (listing 4.5) serves as the user interface of the framework. It is responsible for parsing the DSL and managing the instances of the `Simulator` and `Evaluator` components. Users interact with the framework primarily through the functions provided from this component, for simulating datasets, evaluating models, exporting data, and generating visualizations.

As shown in listing 4.5, the `mestDS` class contains two lists: `simulators` and `evaluators`, which store instances created from DSL configurations. It also defines several utility functions. The `simulate` method iterates over the `Simulator` instances and calls their internal simulation logic, as exemplified in listing 4.5. Similarly, the

`evaluate` method handles model evaluation across multiple datasets. The methods `to_csvs` and `plot_data` are provided to support data export and visualization.

Loose Coupling

Loose coupling can be defined as a framework that connects otherwise autonomous components [20], or components that depend on each other to the least extent practicable [32]. In a loosely coupled system, components are generally cohesive and take on well-defined responsibilities. This allows the system to rely on stable interfaces that limit the exposure of implementation details, as described by Mankovski et al. [20].

During the development of the framework, loose coupling principles have been applied. The components `Simulator` and `Evaluator` have been developed to work autonomously, with their respective responsibilities. Due to how the DSL parsing is carried out in the `mestDS` component, the `Simulator` component is quite dependent on the `mestDS` component. On the other hand, the `Evaluator` component can work well independently of the `mestDS` component.

This modular structure enables users to utilize the framework in varied ways depending on their specific needs, such as exclusively for data simulation, model evaluation, or as a complete pipeline.

```
class mestDS:
    simulators: list[Simulation]
    evaluators: list[Evaluator]

    def __init__(self, dsl_path=None):
    def simulate(self):
        for simulator in self.simulators:
            simulator.simulate()
        return self.simulators
    def evaluate(self, simulations=None, path=None, report_path="reports/")
    def plot_data(self, folder=None, dont_show=[]):
    def to_csvs(self, folder):
```

Listing 4.5: class mestDS

Table 4.1: Methods/Attributes of the `mestDS` component

Method/Attribute	Description
`simulators`	List of `Simulator` instances.
`evaluators`	List of `Evaluator` instances.
`simulate(self)`	Calls the `simulate()` method on all `Simulator` instances. Returns the simulators with generated data.
`evaluate(self, simulations=None, path=None, report_path="reports/")`	Evaluates models using the `Evaluator` instances on simulated datasets or CSV files. Saves the results as reports in the given output folder.
`plot_data(folder, dont_show)`	Generates plots of the simulated datasets, with options to hide certain lines.
`to_csvs(folder)`	Saves the simulated datasets to CSV files in the specified folder.

4.4 Simulator

The `Simulator` component (listing 4.6) is responsible for simulating climate-health time-series datasets based on configurations defined in the DSL. Each instance of `Simulator` includes a set of attributes that control how the simulation behaves.

The attributes `id`, `name`, and `description` are used for identifying and documenting the simulation. The `id` is a user-defined integer used to reference the simulator within the DSL (section 4.2.1), while `name` and `description` help distinguish between different simulations, especially in larger projects or when reviewing evaluation reports.

Temporal aspects of the simulation are controlled using the `time_delta`, `length`, and `start_date` attributes. These define the simulation's temporal granularity (e.g., daily, weekly, or monthly), total duration, and the initial date of the time series.

To support simulations across multiple geographic areas, the `regions` attribute holds a list of `Region` instances (section 4.10). These provide region-specific properties, such as seasonality or neighboring relationships.

The attributes `x` and `y` define the input (features) and output (targets) of the dataset. Each is a list of `Variable` instances (section 4.11) used to simulate values for different features.

In some scenarios, users may define reusable value lists in the DSL for use across multiple features. These are stored in the `public_lists` dictionary upon DSL parsing.

The actual simulated data is stored in the `data` attribute, a nested dictionary whose structure is explained in section 4.4.1. Lastly, the `current_i` and `current_region` attributes are used internally during simulation to keep track of the current time step and region.

```
class Simulator:
    id: int
    name: str
    description: str
    time_delta: Literal["D", "W", "M"] = "W"
    length: int = 100
    start_date: datetime.date = datetime.date(2024, 1, 1)
```

```
regions: list[Region]
x: list[Variable]
y: list[Variable]
public_lists: dict[str, Any]
data: Dict[str, Dict[str, list[float]]]
current_i: int
current_region: str
```

Listing 4.6: class Simulator

Table 4.2: Attributes of the `Simulator` component

Attribute	Description
`id`	User-defined identifier used in the DSL to reference the simulator.
`name`	Short descriptive name of the simulation.
`description`	Brief explanation of the simulation's purpose or characteristics.
`time_delta`	Temporal resolution of the simulation: daily (D), weekly (W), or monthly (M).
`length`	Number of time steps (e.g., days, weeks, months) to simulate.
`start_date`	Starting date of the simulation.
`regions`	List of `Region` instances specifying regional properties.
`x`	List of input (feature) variables.
`y`	List of output (target) variables.
`public_lists`	Dictionary of shared lists defined in the DSL, useful for feature definitions.
`data`	Dictionary storing the simulated dataset (section 4.4.1).
`current_i`	Current time index during simulation.
`current_region`	Current region being simulated.

4.4.1 Data Structure

The structure of the synthetic dataset follows the same format as used in `chap-core` (section 2.8). The top-level keys represent region names. For each region, the value is a dictionary containing lists for every feature—such as `rainfall`, `population`, or `disease_cases`—where each list contains values for sequential time steps.

Listing 4.7 illustrates a simplified example of such a dataset. In this case, there are two regions: `Masindi` and `Buikwe`, each with data for two time steps (e.g., two months). As expected, each feature list contains two values, corresponding to those time periods.

```
{
  'Masindi': {
    'time_period': ['2024-01', '2024-02'],
    'rainfall': [0, 10],
    'population': [996.36, 996.67],
    'mean_temperature': [25.48, 27.34],
    'disease_cases': [0, 10]
  },
```

```
  'Buikwe': {
    'time_period': ['2024-01', '2024-02'],
    'rainfall': [0, 5],
    'population': [20006.54, 20057.23],
    'mean_temperature': [27.45, 27.78],
    'disease_cases': [167, 170]
  }
}
```

Listing 4.7: Data structure of synthetic dataset

4.5 Evaluator

The `Evaluator` component (listing 4.8) is responsible for evaluating models based on evaluation configurations defined in the DSL.

A model is evaluated on a number of datasets passed to the evaluation method of the `mestDS` component. The results of the evaluation from these datasets are stored in the attribute `results`, which holds a list of `Result` instances (see **??**).

Model execution is handled by the `runner` attribute, an instance of `ModelRunner`, which is initialized when the `Evaluator` is created. This object encapsulates the logic for training and testing the model, and returns plots and metrics of the results. Further details about this class are provided in Section 4.9.

In some cases, models may have requirements related to time granularity or minimum time series length. To accommodate this, the `Evaluator` allows for overriding the attributes of a `Simulator` via the optional `time_delta` and `sim_length` attributes. If specified, these override the corresponding settings in the `Simulator` instances used during evaluation.

```
class Evaluator:
    results: list[Result]
    runner: ModelRunner
    time_delta: Literal["D", "W", "M"] | None = None
    sim_length: float | None = None
```

Listing 4.8: class Evaluator

Table 4.3: Attributes of the `Evaluator` component

Attribute	Description
`results`	A list of `Result` instances, each representing the evaluation output for a dataset.
`runner`	An instance of `ModelRunner`, responsible for training and testing the model.
`time_delta`	Optional override of the time resolution used in the simulator during evaluation (e.g., "D", "W", "M").
`sim_length`	Optional override for the length of the simulated time series used during evaluation.

4.5.1 ModelRunner

When an `Evaluator` instance is created, an instance of `ModelRunner` is also initialized. As briefly mentioned in the previous section, this class holds the logic for training and testing a model, as well as returning plots and metrics based on the evaluation results. The attributes of a `ModelRunner` instance define how the evaluation is carried out.

The attribute `model_path` specifies which model should be evaluated. This can either be a local file path or a GitHub repository URL.

The attributes `prediction_length`, `n_test_sets`, and `stride` define how the test sets are constructed:

- `prediction_length` determines how long each test set is.
- `n_test_sets` defines how many test sets the model should be evaluated on.
- `stride` controls how far each test set moves forward from the previous one.

Note that the appropriate values for `prediction_length` can vary depending on the model's expected time granularity. For instance, a weekly model might have a prediction length of 12 (i.e. 12 weeks), while a monthly model might require a different value.

The attribute `metrics` holds a list of evaluation metrics to use for this particular instance. The framework provides a pool of commonly used metrics to choose from.

In cases where the dataset is large, full-length plots can become hard to interpret. The `plot_length` attribute addresses this by specifying how many of the most recent data points should be plotted. For example, if set to 100, only the last 100 data points will be shown in the plot.

Some models provide additional user options in their `MLProject` files, such as time lags or hyperparameters. The attribute `user_options` allows these to be overridden.

```
class ModelRunner:
    model_path: str
    prediction_length: int
    n_test_sets: int
    stride: int
    metrics: list[str]
    plot_length: int
    user_options: dict | None
```

Listing 4.9: class ModelRunner

Table 4.4: Attributes of the `ModelRunner` component

Attribute	Description
`model_path`	Path or URL pointing to the model.
`prediction_length`	Number of time steps per test set.
`n_test_sets`	Number of test sets to use during evaluation.
`stride`	Step size between the start of each test set.
`metrics`	List of metric names to evaluate model performance.
`plot_length`	Number of data points to display in plots.
`user_options`	Optional dictionary to override user options from the model's MLProject file.

4.6 Supporting Components

In addition to the core components, the framework includes several supporting components that contribute to specific aspects of simulation and evaluation. These components are briefly introduced in this section.

Region

The `Region` component (listing 4.10) defines metadata about a geographic region involved in the simulation. It contains user-defined identifiers such as `name` and `region_id`, along with relational and behavioral attributes.

The `neighbour` attribute is a list of region IDs indicating spatial proximity, enabling interaction between regions in multi-region simulations. The `seasons` dictionary provides a flexible mechanism for defining seasonal patterns, such as rain seasons, heat waves, or climate health-specific events like outbreaks, specific to each region.

```
class Region:
    name: str
    region_id: int
    seasons: dict[str, Any]
    neighbour: list[int]
```

Listing 4.10: class Region

Variable

The `x` and `y` attributes of a `Simulator` instance are lists of `Variable` objects. A `Variable` represents either an explanatory variable (`x`) or a target variable (`y`) in the time series dataset. Each instance contains a variable name, a user-defined function (as a string), and a dictionary of parameters passed to that function.

```
class Variable:
    def __init__(self):
        self.name: str = ""
        self.function: str = ""
        self.params: dict[str, Any] = {}
```

Listing 4.11: class Variable

Result

When the `ModelRunner` returns the result from a model evaluation for a single dataset back to the `Evaluator`, it does so as an instance of the `Result` class. As shown in listing 4.12, this class holds metadata such as the dataset or simulation name and an optional description. It also includes visualizations of the results for each region and a list of performance metrics.

```
class Result:
    name: str
    description: str
    plots: list[matplotlib.figure.Figure]
    location_metrics: list[LossMetrics]
```

Listing 4.12: class Result

4.7 Pipeline - from DSL to Model Insights

With the frameworks architecture in mind, we can now have a look at the pipeline from defining `Simulator` and `Evaluator` configurations in the DSL, to gaining model insights from the model evaluation report that is generated:

1. Define the `Simulator` and/or `Evaluator` configurations in the DSL, as section 4.2 explains.

2. Initialize a `mestDS` class, with the path of the DSL as an argument. This will parse the DSL and create instances of the `Simulator` and/or `Evaluator` classes.

3. If you have defined `Simulator` configurations in your DSL, run `mestDS`'s function `simulate()`. This function returns a list of `Simulator` instances, with a populated `data` attribute.

4. If you have defined `Evaluator` configurations in your DSL, run `mestDS`'s function `evaluate()`. Pass the datasets you wish to evaluate the model on as an argument. This could either be the datasets you simulated in step 3 (by passing their corresponding `Simulator` instances), or a local file path to either one CSV file or a folder containing several CSV files. This function generates a model evaluation report to the local file path that you passed as an argument, along with the datasets for the evaluation.

4.7.1 Simulation Process

With an understanding of the framework and the overall pipeline in place, we now take a closer look at the simulation process itself.

Synthetic datasets are generated by calling the `simulate()` method of the `Simulator` class.

The simulation proceeds over a number of time steps, iterating through a list of regions. For each region and time step, the simulator processes all variables listed in the attributes `x` and `y`. The value of each variable is computed by executing its associated function—defined in the DSL—and the result is stored in the `Simulator`'s data structure.

Listing 4.13 presents a simplified pseudo code version of the `simulate()` function.

```
function simulate():
    initialize the data structure
    determine time delta based on configuration

    for i in 1 to simulation length:
        compute the current date based on i and time delta
        for region in regions:
            add current date to region's time period list
            for variable in x:
                compute and store value using associated function
            for variable in y:
                compute and store value using associated function
```

Listing 4.13: Pseudocode of Simulator.simulate()

4.7.2 Evaluation Process

Moreover, the evaluation process is designed to be flexible, allowing models to be tested either on synthetic data produced by simulators or on datasets provided as CSV files.

Listing 4.14 outlines the pseudo code for the `evaluate()` function. When a list of simulators is provided, the evaluator optionally overrides their `time_delta` and `length` attributes based on the DSL configuration of associated `Evaluator` instance. If any of these attributes are overridden, the `simulate()` function is re-executed to generate a new time series. The model is then evaluated against each simulated dataset, and the resulting performance metrics and plots are stored. If the simulator contains a description, it is appended to the corresponding result.

Alternatively, if a file or folder path is provided, the evaluator reads one or more CSV files directly. If the path points to a single CSV file, that file is used for evaluation. If the path points to a folder, all CSV files within it are iterated over and used for evaluation. The results are appended accordingly. If neither a simulator nor a valid path is provided, an error is raised.

After all evaluations are complete, a report containing the results is generated and stored at the specified output path. The report contains a table of all performance metrics for all regions, and a plot of the predictions per region, for all datasets.

```
function evaluate(report path, simulators, local file path)
    if simulators:
        for simulator in simulators:
            if override time delta:
                simulator.time_delta = new time_delta
            if override simulation length:
                simulator.sim_length = new simulation length
            if override time delta or simulation length:
                simulator.simulate()
            evaluate model with simulator and add result to 'results'
            if simulator has description:
                add description to the last object in 'results'
    elif path:
        if path is a file and ends with '.csv':
            evaluate model with csv file and add result to 'results'
        elif path is a folder:
            for filename in csv files in folder:
                evaluate model with csv file and add result to 'results'
        else:
            raise error
    generate and store model evaluation report to 'report path'
```

Listing 4.14: Pseudocode of Evaluator.evaluate()

4.8 Summary

This chapter gives both a general overview and detailed description of the framework's architecture and pipeline. The framework `mestDS` consists of a DSL and the core components: `mestDS`, `Simulator` and `Evaluator`.

The DSL enables flexibility, expressiveness and effectiveness in defining configurations for synthetic datasets and model evaluations. Although the framework is designed for climate health time series, in practice, the DSL can simulate any kind of time series, due to the user defined variables. Furthermore, due to the DSL's inheritance (section 4.2.1), multiple `Simulator` configurations can be defined, with minor code. Public functions also enables the re-use of functions for multiple configurations and variables.

The emphasize on loose coupling in the framework allows the components `Simulator` and `Evaluator` to work independently, increasing the framework's flexibility as well. If a user wish to only simulate a synthetic dataset or evaluate a model, the loose coupling enables this.

Chapter 5

Implementation

While chapter 3 captured how the research was conducted methodologically, and chapter 4 captured the frameworks architecture and pipeline, this chapter aims at elaborating how the research and the implementation of the framework was conducted practically.

As stated in chapter 3, the research followed an iterative design process. This chapter goes through the iterations of development, feedback and evaluation, and literature review. It starts by delving into the initial pilot projects conducted at the outset of the thesis, which built the foundational knowledge regarding data simulation and the DGP of VBD, as well as the foundational code base for the rest of the research. Furthermore, the chapter delves into the following iterations after the pilot projects, where many of the core design principles was implemented, such as the DSL. Lastly, the chapter elaborates on the final evaluation phase of the framework. During this phase, multiple weaknesses was addressed and improved. Although the development process did not follow formally defined phases, this chapter organises it into logical stages and presents them in chronological order.

5.1 Phase 1 - Pilot Projects

At the outset of this thesis, a series of pilot projects were proposed by the supervisor to provide a practical foundation for understanding data simulation, particularly in the context of vector-borne diseases. The three pilot projects built upon each other, and incrementally increased the complexity of the data simulation process.

The pilot projects served as an introduction to both synthetic data generation and the domain-specific modeling of vector-borne disease transmission. A literature review was conducted in parallel, facilitating a deeper understanding of relevant topics such as climate-health relationships, disease modeling, and the role of synthetic datasets in model development and evaluation.

Pilot Project 1

The first pilot project aimed to generate a minimal synthetic dataset in CSV format, containing the columns week number, rainfall, and sickness, where sickness depended directly on the explainable variable rainfall.

The Python script assigned random initial values for rainfall and sickness in the first week. For subsequent weeks, rainfall was generated as a random integer between 0 and 100. The sickness level was then adjusted from the previous week according to predefined

rainfall intervals. For example, if rainfall was 70, sickness increased by 5 units from the previous week; if rainfall was 5, sickness decreased by 10 units.

No attempt at modularization or fragmentation of the script was made, resulting in *smelly* code: poor readability, hard to debug, and difficult to scale.

Pilot Project 2

Building on the previous simulation, a second explanatory variable, temperature, was introduced. Rain seasonality was also introduced in this project. The weighted dot product of rainfall and temperature decided the increase or decrease of sickness, with the same interval based approach.

$$[rainfall, temperature]^T [rainfall'sweight, temperature'sweight]$$

The *smelly* code from project 1 was addressed to some extent. Code was fragmented into separate functions, improving readability, scalability and debugging. Simulation length and the option for disabling rain season was parameterized, giving the user more flexibility.

Pilot Project 3

The third project moved from simulation to data analysis. Simulating datasets over multiple years enabled the computation of weekly averages across years, revealing seasonal trends when plotted. For example, a three year long simulation allowed direct visualization of repeating yearly sickness patterns.

Although this project did not expand the simulation process itself, the hands-on exploration of seasonality provided a deeper understanding of how seasonal patterns appear in time-series data and its potential relevance to VBD.

5.2 Phase 2 - Python Library, Parameterization, Multiple Simulations, and chap-core

After the pilot projects initiated by the supervisor, the development continued on one's own initiative. As stated in chapter 3, the iterative development cycles were guided by expert feedback, evaluation of the framework and literature review.

To start, the resulting framework from the pilot projects was encapsulated as a Python library. This marks the shift towards a more *API-focused* approach.

Moreover, a main priority of the framework, is that is works with chap-core models. These models rely heavily on chap-core functionality and data structures. Hence, chap-core's data structure was adopted for the simulated data. Chap-core being a Python library, allowed us to simply import the necessary data classes.

Enhancing user flexibility is a main theme throughout the entire development process. During this phase, multiple variables was added and parameterized. Start date and region name of the simulation was parameterized. Later on in the development, allowing multiple regions was also introduced. Furthermore, the option of choosing what time granularity the simulation had was introduced, allowing the user to chose between daily, weekly and monthly time series. Some models require different time series interval, so this was crucial for the integration with chap-core models.

As stated in section 2.5, a model should be evaluated on multiple datasets. To address this, a method for simulating multiple synthetic datasets with varying characteristics was

developed. An important part of this change was the introduction of a revised sickness model, i.e., the method used to calculate the number of disease cases each week.

The previous approach calculated a weighted dot product of rainfall and temperature to determine the week-to-week change in disease cases. While this method produced a clear relationship between the explanatory variables and sickness levels, it exhibited a problem with *stationarity*. Specifically, if the simulation started at a high value (e.g., 1000 cases) and ran for a long duration, the case count tended to drift upwards or downwards over time, producing an unrealistic trend. As the supervisor pointed out, the disease level should fluctuate around a stable range rather than steadily increasing or decreasing. Additionally, the previous approach was not robust to changes, as the model used hard coded values when indexing the correct interval, as seen in listing 5.1

```
def get_sickness(sickness, input, weight):
    sum = np.dot(input, weight)
    if sum > 65.2:
        sickness = sickness + random.randint(6, 10)
    elif sum >= 49.2:
        sickness = sickness + random.randint(0, 5)
    elif sum < 33.2:
        sickness = sickness + random.randint(-10, -6)
    elif sum < 49.2:
        sickness = sickness + random.randint(-5, 0)
    if sickness < 3:
        return random.randint(0, 3)
    else:
        sickness = sickness + random.randint(-3, 3)
        return sickness
```

Listing 5.1: Sickness model version 1

To address this, a new sickness model was introduced:

$$S_t = S_{t-1} + \left\lfloor \mathcal{N}\left(\frac{D}{M} - sp,\ sa\right) \cdot si \right\rfloor \tag{5.1}$$

Here:

- S_t is the sickness level at week t.
- D is the weighted sum of explanatory variables.
- M is the maximum possible dot product value for normalization.
- sp defines the baseline around which sickness levels fluctuate.
- sa controls variability in weekly changes.
- si adjusts the magnitude of changes in case numbers.

Altering the values of sp, sa, and si changes the characteristics of the simulation. The newly introduced method for simulating multiple datasets included parameters for simulation length, regions, and start date. The parameters sp, sa, and si were also made configurable, allowing the user to pass a list of values for each. The method iterated over all possible combinations of the provided values, generating a separate dataset for each configuration. For example, if three values were passed for each parameter, the method would produce a total of $3 \times 3 \times 3 = 27$ datasets, each with different characteristics.

5.3 Phase 3 - Modularization and DSL

The development process continued with modularization of the framework. In phase 2, the framework had been fragmented into separate functions. To further improve code readability and maintainability, the framework was reorganized into classes. Specifically, code relevant to a single simulation was encapsulated in the class `Simulation`, while code handling multiple simulations was encapsulated in the class `MultipleSimulations`.

The sickness model introduced in the previous phase initially showed promise, allowing the generation of simulations with varying characteristics. Initial tests also suggested that it addressed the problem of *stationarity*. However, it later became apparent that the issue persisted, necessitating a new model. At this point, a linear sickness model was developed:

$$S_t = \beta_0 + \beta_1 \cdot \text{rainfall}_t + \beta_2 \cdot \text{temperature}_t \tag{5.2}$$

where β_0 is the intercept (baseline sickness level when rainfall and temperature are zero), β_1 is the coefficient for rainfall, and β_2 is the coefficient for temperature. Later, an additional explanatory variable, *lagged sickness*, along with its corresponding coefficient, was incorporated into the model, introducing a degree of autoregressiveness. The coefficients in this model replaced the parameters discussed in the previous phase, allowing the user alter simulations based on coefficients of the explainable variables.

As the simulation process developed, flexibility increased, but this also led to greater complexity and a higher number of parameters. Additionally, the iterative approach to simulating multiple datasets sometimes produced unintentional datasets, i.e. datasets that did not meaningfully contribute to model evaluation. A crucial aspect of model evaluation is selecting datasets that are informative and relevant [26]. This highlighted the need for a framework that allows users to define simulations intentionally and efficiently. A DSL was proposed as a solution to this problem.

Listing 5.2 shows the original DSL, where users directly specify coefficient values for each simulation rather than passing lists of values. While this approach did not alter the sickness model or other aspects of the simulation, the DSL acted as a thin layer over the framework, increasing transparency and interpretability of the simulations.

In Listing 5.2, multiple simulations are defined in two steps:

1. A base instance of `Simulation` is defined. Other simulation instances are derived from this base. In this example, the time granularity is set to days and the simulation length is set to 500.

2. The specific simulations to be executed are listed under the keyword `sims`. Each entry represents a simulation, with default coefficient values overridden as needed. In this example, one simulation is defined where the temperature weight is higher than the rainfall weight, another where rainfall weighs more than temperature, and a third where both weights are equal.

```
simulation:
  base:
    time_granularity: "D"
    simulation_length: 500
  sims:
    - beta_rain: 0.5
      beta_temperature: 0.9
```

```
  - beta_rain: 0.9
    beta_temperature: 0.5
  - beta_rain: 0.5
    beta_temperature: 0.5
```

Listing 5.2: DSL v.1 - defining multiple simulations with DSL

5.4 Phase 4 - User-defined features

Although the simulation framework at this stage was capable of producing complex simulations with diverse characteristics, users were still constrained to a set of predefined functions, one each for rainfall, temperature, and disease cases.

Expert feedback highlighted this limitation, leading to the design of a more flexible DSL. After several iterations, the framework evolved into a version where users could define their own simulation features directly. This shifted the framework from a restricted but efficient system toward one that prioritizes flexibility and expressiveness over efficiency. In principle, this new approach enables the simulation of any possible time series.

Listing 5.3 illustrates DSL version 2. The overall structure is much like version 1, but with key differences:

1. The `base` is now referred to as `model`. Unlike in version 1, the base model is itself part of the set of simulations to be run. Under the `model` keyword, variables such as simulation name, time granularity, and simulation length are defined.

2. Under `features`, each feature is given a name and a user-defined function. The DSL embeds Python code to specify how each feature is calculated per time step, offering complete control over the simulation process. Regions are also defined here.

3. Additional simulations appear under the `simulations` keyword. Each entry defines a new simulation, with optional overrides to the base model's parameters or feature functions.

```
model:
  simulation_name: "sim 1"
  time_granularity: "M"
  simulation_length: 100
  features:
    - name: "rainfall"
      function: |
        def get_rainfall(region, i):
          return 4 if i == 50 else 0
    - name: "mean_temperature"
      function: |
        def get_temperature(i):
          return 4 if i == 50 else 0
    - name: "white_noise"
      function: |
        def get_white_noise():
          return 0
    - name: "disease_cases"
      function: |
        def get_disease_cases(i, rainfall, mean_temperature):
          population = 100
          poisson_rate = sum(
              np.roll(cov, lag)[i]
```

```
            for cov, lag in zip([rainfall, mean_temperature], [3, 3])
        )
        cases = (1 / (1 + np.exp(-poisson_rate))) * population
        return min(np.random.poisson(cases), population)
  regions:
    - name: "Finnmark"
      region_id: 1
      rain_season: []
      neighbour: []
simulations:
  - simulation_name: "sim 2"
    features:
      - name: "white_noise"
        function: |
          def get_white_noise(i, white_noise, rainfall, mean_temperature):
            from sklearn.preprocessing import StandardScaler
            wn = np.random.normal(0, 0.2)
            for cov, lag in zip([rainfall, mean_temperature], [3, 3]):
              lagged = np.roll(cov, lag)
              scaled = StandardScaler().fit_transform(lagged[1:].reshape(-1, 1))
              wn += scaled.flatten() * 0.5
            return wn[-1]
      - name: "disease_cases"
        function: |
          def get_disease_cases(i, rainfall, mean_temperature, disease_cases, white_noise):
            population = 100
            cum_noise = np.cumsum(white_noise)
            prev_cases = cum_noise[i - 1] if i > 0 else 0
            adjusted = np.insert(cum_noise, 0, 0) - 0.5
            scaled_cases = (1 / (1 + np.exp(-adjusted))) * population
            return np.int32(scaled_cases[-1])
```

Listing 5.3: DSL v.2 - defining multiple simulations with DSL

5.5 Phase 5 - Evaluation Module

This section discusses the implementation of the evaluation module. In practice, the development of this module occurred in parallel with versions two and three of the DSL, but it has been separated into its own phase for clarity and narrative purposes.

A key part of the framework is the integration with `chap-models`, enabling users to evaluate models on simulated data. To maintain a consistent structure, this functionality was encapsulated in a separate class, `Evaluator`. During this phase, the classes `MultipleSimulations` and `Simulations` were also renamed to `mestDS` and `Simulator`, respectively. Greater emphasis was placed on naming components and variables thoughtfully, as weaknesses in naming became apparent during framework demonstrations. For example, naming the components `Simulator` and `Evaluator`, rather than `Simulation` and `Evaluation`, emphasizes that the classes act as agents for their functionality rather than representing its results.

The `chap-core` repository already provided functionality to fetch models from GitHub URLs and run models. During implementation, several issues with `chap-core` were encountered: older chap-models required earlier versions of chap-core, and newer models required updated versions. Furthermore, chap-core's model execution demanded strict data formatting, particularly for weekly and monthly dates, which deviated from common conventions.

As the framework aims to support robust and comprehensive model evaluations, results must be stored persistently and presented clearly. Functionality for generating reports with plots and tables of performance metrics was developed. Multiple variations were tested, including color schemes, plot types, and page orientations.

Stress testing the framework (section 5.6), where multiple models were evaluated across various simulations, revealed several weaknesses. As the number of simulations increased, keeping track of their intended purpose became increasingly difficult. Adding descriptions to each simulation and including it in the report enhanced keeping track for the user and improved interpretability for external evaluators without prior knowledge of the simulation process.

The final version of the evaluation component is further discussed in section 4.8.

5.6 Phase 6 - Framework Stress Testing

In the final phases of development, the framework was subject to stress testing through comprehensive model evaluations. The primary objective of this testing was to identify weaknesses in the framework and guide subsequent improvements.

Evaluating multiple models required generating numerous simulated datasets covering the aspects of model evaluation discussed in section 2.5. Over time, the DSL grew increasingly complex, resulting in decreased readability and navigability. To address this, a new iteration of the DSL introduced a public space for reusable functions and lists, and allowed simulations to inherit from simulations other than the base model. The new *inheritance* feature meant that a base model was no longer needed, and could therefor be removed from the DSL's structure. The final design of the DSL is discussed further in section 4.2.

Another weakness identified during this phase was the tight coupling between simulations, evaluations, and generated reports. To improve flexibility, loose coupling was incorporated, enabling users to evaluate models on datasets beyond those generated by the framework. Additionally, report generation, which had previously depended on the `Simulator` class, was refactored to enhance modularity and independence.

Chapter 6

Experiments and Results

A central part of Design Science research is the evaluation of the proposed design. As shown in the previous chapter, the artifact underwent several iterations of refinement, guided by expert feedback, literature review, and continuous assessment. The final phase of development was dedicated to a more systematic evaluation, where the framework was used for its intended purpose: simulating time series and evaluating models. This chapter presents these experiments, illustrating how the framework adds value to both data simulation and model evaluation, and thereby demonstrating the outcomes of this thesis.

6.1 MIMES

This section discusses model evaluations based on the *MIMES* approach, discussed in section 2.6.

6.1.1 MIMES 1 - Learning Seasonality

VBD dynamics often exhibit strong seasonal patterns [24]. Evaluating whether models can capture such patterns is therefore an important step. Since seasonality is relatively predictable [15], this experiment serves as a *sanity check*: a model that fails to capture seasonality in simplistic simulations is unlikely to perform well on more complex, real-world datasets. Beyond confirming predictive ability, this test also provides insights into learning requirements, such as the length of training series and the number of seasonal cycles required for reliable performance.

Set Up

In order to evaluate a model's ability to capture seasonal patterns, a DSL configuration was defined. Appendix B lists the complete DSL for this test:

- Under the `public` key, all functions used in the simulations were defined.
- In addition, two lists, `annual_spikes` and `biennial_spikes`, were created. The list `annual_spikes` contains equally spaced values ranging from 5 to 125, with a 12-month interval, thereby representing annual seasonal rainfall spikes. Similarly, `biennial_spikes` contains equally spaced values between 5 and 113, with a 24-month interval, representing biennial rainfall spikes. These lists are included in the parameter configuration and used by the function `get_rainfall_spike`. If

the current time step is included in the list passed to the function, the function returns a value of 50, otherwise, it returns a random value between 0 and 2. The introduction of randomness prevents model errors that occur when values remain constant over time. Since chap-models require a minimum time series length, evaluating performance on only a few seasonal cycles necessitates an alternative design. For this purpose, a biennial rainfall spike list was incorporated, allowing models to be trained and evaluated on just two seasonal cycles

- As the DSL configuration is designed to evaluate chap-models, it also requires the variables `population` and `temperature`. Both of these are generated using the function `get_flat_value`, which accepts two parameters, `mean` and `randomness`, and returns values fluctuating around the specified mean with added variation.
- The number of disease cases is derived using the function `get_disease_cases_with_lag`, which takes rainfall and a lag parameter as inputs. This function simply returns the rainfall value from n months earlier, where n is the specified lag, provided that the time series is sufficiently long.
- All simulations are defined under the keyword `simulations`. The initial simulation specifies an identifier, name, and description, as well as features, time delta, and other parameters, as shown in Listing 6.1. The simulation length is set to 60 months and includes four seasonal rainfall spikes, as suggested by the variable `name` in the DSL configuration.
- Additional simulations were also defined in the DSL. As shown in Listing 6.2, these simulations inherit from the initial configuration. They extend the simulation length, thereby increasing the number of seasonal rainfall spikes, and in some cases replace the `annual_spikes` parameter with `biennial_spikes` to evaluate biennial periodicity.
- Finally, the DSL also specifies configurations for the models to be evaluated, as shown in Listing 6.3.

```
simulators:
  - id: 1
    name: seasonal spikes for 60 months (trained on 4 spikes)
    description: Is the model able to detect the lag relationship between r
    time_delta: M
    length: 60
    x:
      - name: rainfall
        function_ref: get_rainfall_spike
        params:
            list: annual_spikes
      - name: population
        function_ref: get_flat_value
        params:
          mean: 1000
          randomness: 5
      - name: mean_temperature
        function_ref: get_flat_value
```

```
        params:
          mean: 27
          randomness: 3
    y:
      - name: disease_cases
        function_ref: get_disease_cases_with_lag
        params:
          lag: 3
```

Listing 6.1: DSL configuration for "MIMES 1 – initial simulation"

```
  - id: 3
    inherit: 1
    name: seasonal spikes for 70 months (trained on 5 spikes)
    length: 70
  - id: 4
    inherit: 1
    name: seasonal spikes for 82 months (trained on 6 spikes)
    length: 82
  - id: 2
    inherit: 1
    name: seasonal spikes for 60 months (trained on 2 spikes, biennial)
    length: 60
    x:
      - name: rainfall
        params:
            list: biennial_spikes
```

Listing 6.2: DSL configuration for "MIMES 1 – other simulations"

```
evaluators:
  - model: https://github.com/dhis2-chap/ewars_template.git
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    metrics: [mse, pocid, theils_u]
    plot_length: 60
  - model: "https://github.com/sandvelab/chap_auto_ewars"
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    plot_length: 60
    metrics: [mse, pocid, theils_u]
  - model: https://github.com/sandvelab/monthly_ar_model
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    metrics: [mse, pocid, theils_u]
    plot_length: 60
```

Listing 6.3: DSL configuration for "MIMES 1 – model configurations"

Results

`chap_auto_ewars` This model produced accurate predictions even with limited training data. When trained on time series containing only two seasonal spikes, it successfully captured the seasonal pattern (Figure 6.1). Accuracy improved as more cycles were available for training (Figure 6.2).

Simulation: seasonal spikes for 60 months (trained on 2 spikes, biennial)

Description: Is the model able to detect the lag relationship between rainfall and disease cases?

Location	mse	pocid	theils_u
Masadi	122.36	35.29	0.66

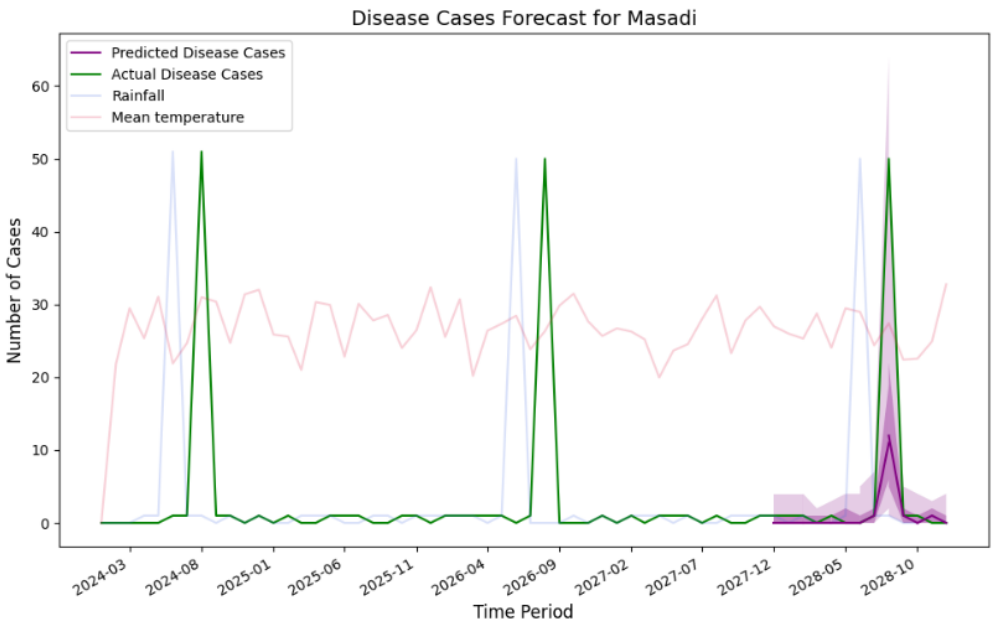

Figure 6.1: MIMES 1 - `chap_auto_ewars` performance on 60-month simulation (2 seasonal spikes).

Simulation: seasonal spikes for 70 months (trained on 5 spikes)

Description: Is the model able to detect the lag relationship between rainfall and disease cases?

Location	mse	pocid	theils_u
Masadi	0.78	23.53	0.05

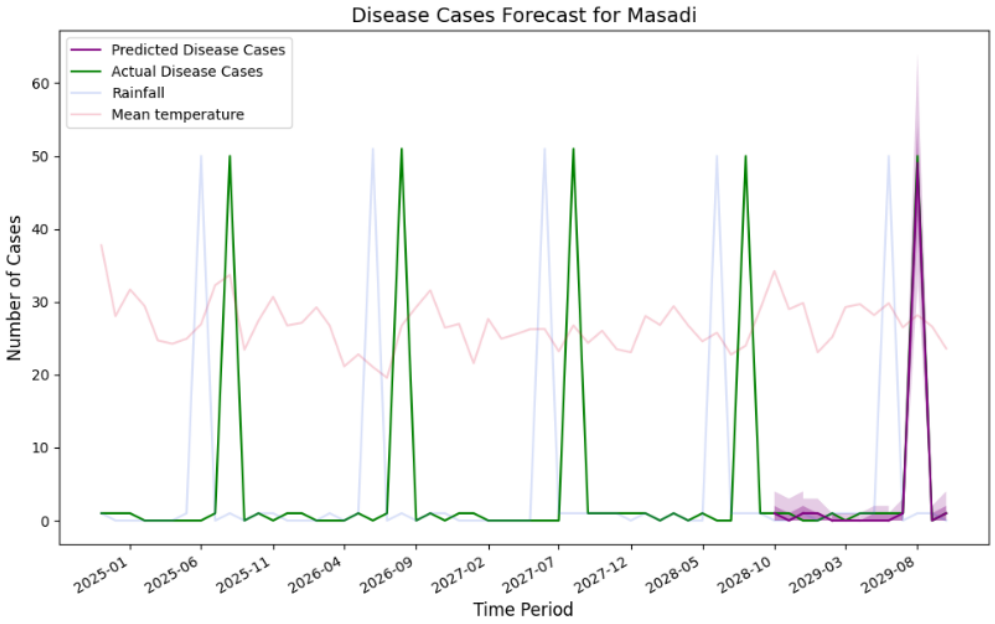

Figure 6.2: MIMES 1 - `chap_auto_ewars` performance on 70-month simulation (5 seasonal spikes).

`ewars_template` This model struggled when trained on shorter time series. For instance, with five seasonal spikes, the predictions were inconsistent (Figure 6.3). However, once the training set included at least six seasonal cycles, the model began to capture the seasonal pattern more reliably (Figure 6.4).

Simulation: seasonal spikes for 70 months (trained on 5 spikes)

Description: Is the model able to detect the lag relationship between rainfall and disease cases?

Location	mse	pocid	theils_u
Masadi	244.57	23.53	0.92

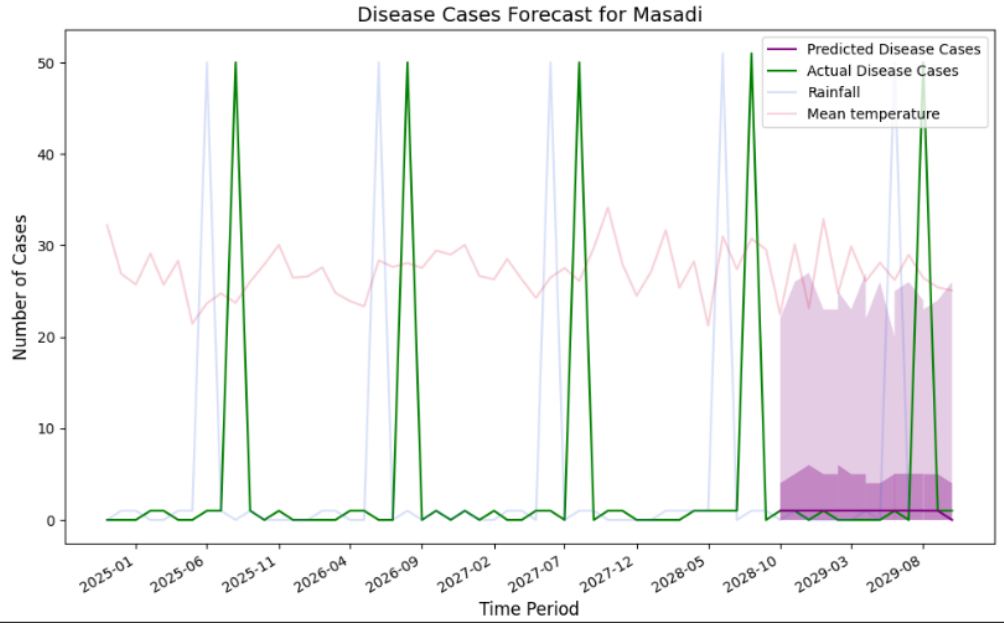

Figure 6.3: MIMES 1 - `ewars_template` performance on 70-month simulation (5 seasonal spikes).

Simulation: seasonal spikes for 82 months (trained on 6 spikes)

Description: Is the model able to detect the lag relationship between rainfall and disease cases?

Location	mse	pocid	theils_u
Masadi	8.92	23.53	0.17

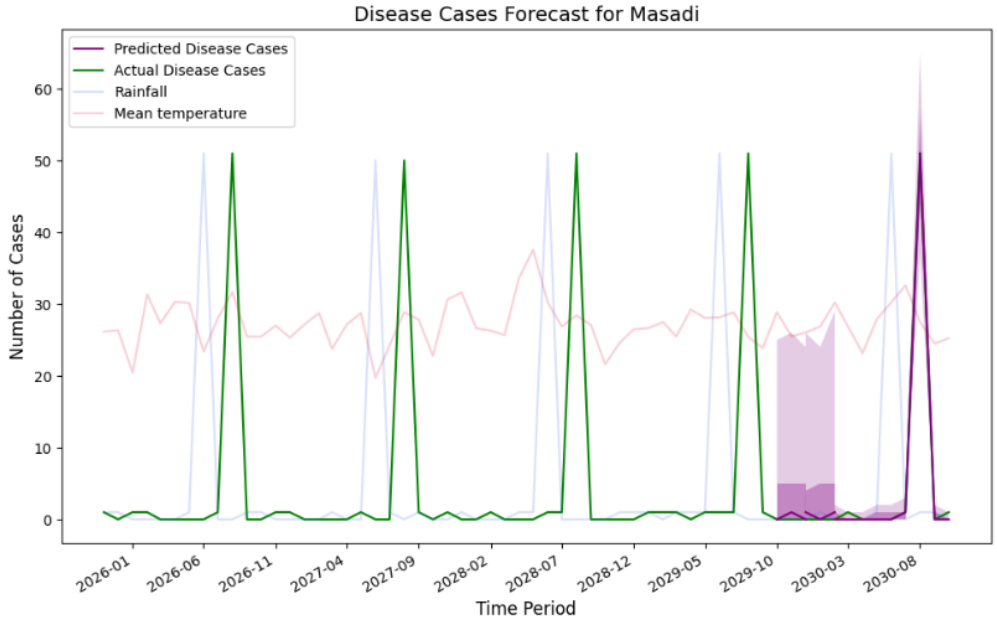

Figure 6.4: MIMES 1 - `ewars_template` performance on 82-month simulation (6 seasonal spikes).

`monthly_ar_model` This model consistently failed to learn the seasonal pattern, even when trained on a considerably longer time series. For example, with thirteen seasonal cycles in the training set, the predictions did not reflect the expected lagged rainfall-disease relationship (Figure 6.5).

Simulation: seasonal spikes for 164 months (trained on 13 spikes)

Description: Is the model able to detect the lag relationship between rainfall and disease cases?

Location	mse	pocid	theils_u
Masadi	199.03	35.29	0.82

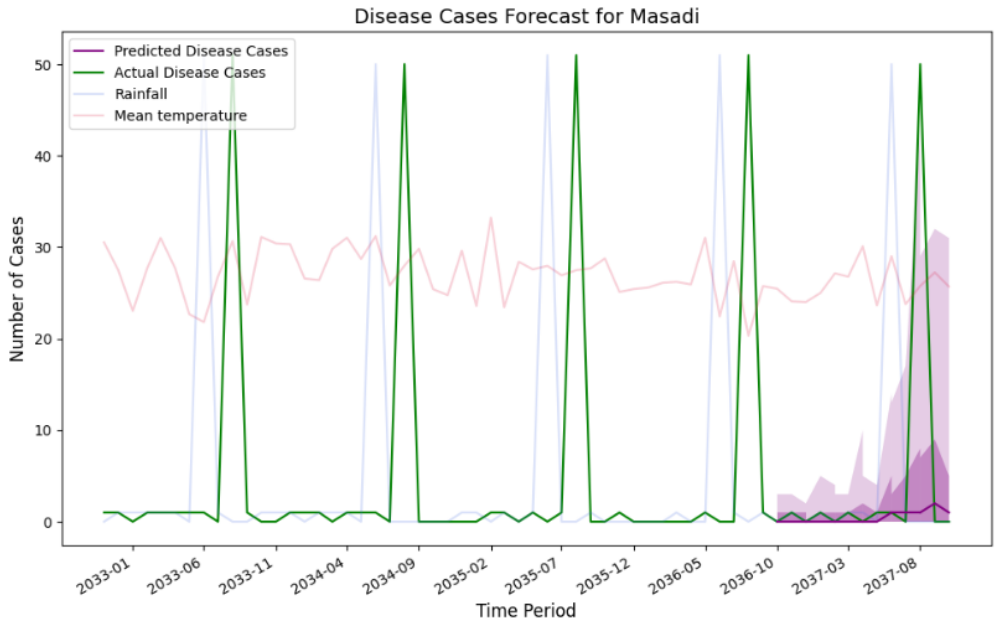

Figure 6.5: MIMES 1 - `monthly_ar_model` performance on 160-month simulation (13 seasonal spikes).

Discussion

These findings underline some important insights. First, it is clear that `chap_auto_ewars` learned the seasonal pattern the easiest, and gives an indication that it would perform better than `ewars_template` and `monthly_ar_model` in regards of predicting seasonal outbreaks in real-world scenarios, especially on smaller training sets.

Second, `monthly_ar_model` did not learn the seasonal pattern at all, even when trained on a 13 year long training set (figure 6.5). This indicates that the model requires large datasets to learn simplistic seasonal mechanisms, which is not ideal in VBD prediction, where data is scarce [15].

6.1.2 MIMES 2 - Learning correlation between lagged rainfall and disease cases

Research shows that rainfall can increase the number of disease cases after a period of time [6]. This test aims at evaluating the models ability to capture the lagged relationship between rainfall and disease cases.

Set up

While the previous test simulated annual and biennial rainfall spikes, this test simulates random rainfall spikes, consequently removing the seasonal aspect of the time series. Hence, the model is required to capture the lagged relationship between rainfall and disease cases, opposed to the seasonality of disease cases.

In order to evaluate the models predictive ability of the given mechanism, small adjustments from the previous *MIMES*'s DSL is required. The lists `annual_spikes` and `biennial_spikes` has been replaced by a single list containing random values ranging from 4 to 600. At the beginning of this *MIMES*, the amount of random rainfall spikes was equivalent to the same amount of seasonal spikes from the previous *MIMES*. However, as the models were not able to capture the simulated mechanism, the number of random spikes increased. In the end, 115 random rainfall spikes was included.

Beside replacing the list of values for rainfall spikes, more simulations were added, simulation names changed, and lengths altered in order to get optimal test sets.

Result

The results demonstrated that the model `monthly_ar_model` required a substantial amount of rainfall spikes in the training set, in order to learn the simulated mechanism, i.e. the lagged relationship between rainfall and disease cases. In fact, the model required 97 rainfall spikes in the training set, before learning the mechanism. Figure 6.6 shows the result from the 420 month long simulation, consisting of 87 random spikes. The plot demonstrates that the model was not able to detect the lagged relationship between rainfall and disease cases. However, the 500 month long simulation, consisting of 99 random spikes, was able to detect this relationship, as figure 6.7 demonstrates.

Simulation: random spikes for 420 months (87 spikes)

Description: Is the model able to detect the lag relationship between rainfall and disease cases?

Location	mse	pocid	theils_u
Masadi	282.03	29.41	0.71

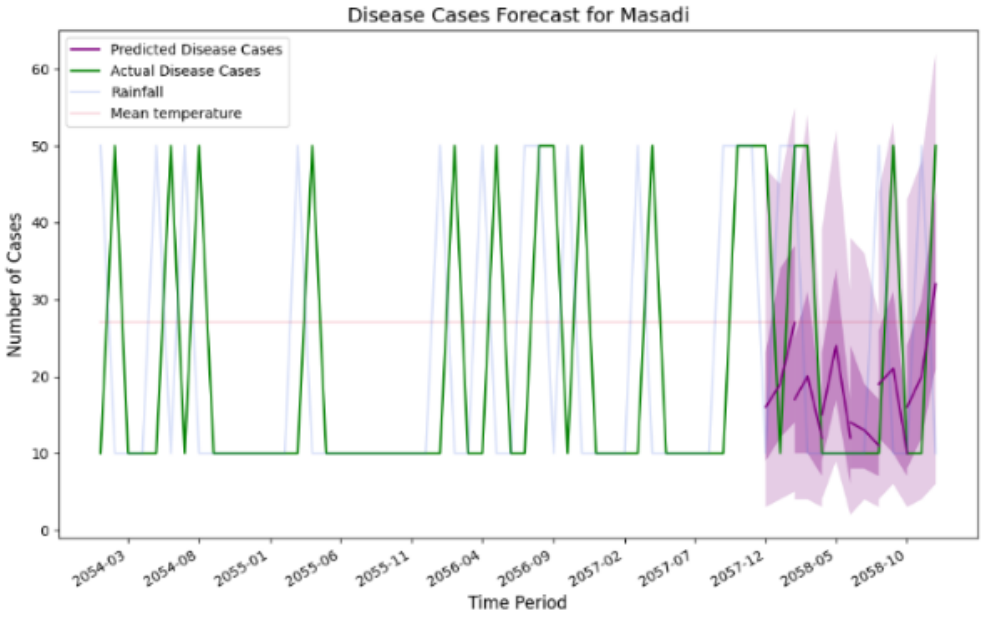

Figure 6.6: MIMES 2 - `monthly_ar_model performance on 420 month long simulation (87 random spikes)`

Simulation: random spikes for 500 months (99 spikes)

Description: Is the model able to detect the lag relationship between rainfall and disease cases?

Location	mse	pocid	theils_u
Masadi	84.78	23.53	0.47

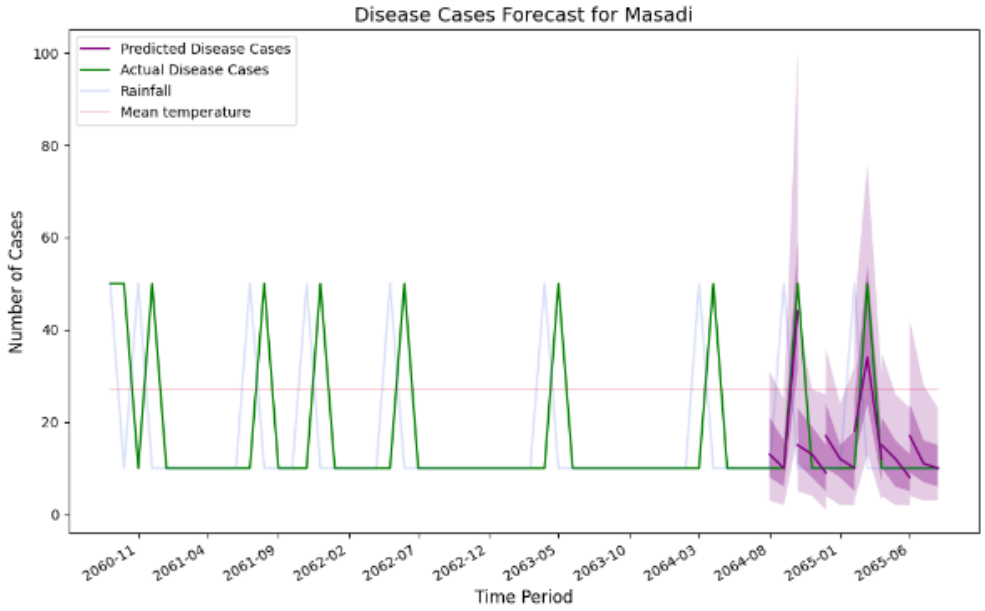

Figure 6.7: MIMES 2 - `monthly_ar_model performance on 500 month long simulation (99 random spikes)`

The models `chap_auto_ewars` and `ewars_template` was not able to learn the mechanism at all. An additional simulation, with 169 total rainfall spikes, was added to further validate this finding.

Discussion

In contrast to the previous *MIMES*, where `monthly_ar_model` failed to capture seasonal patterns, this test showed that `monthly_ar_model` was the only model to learn the lagged relationship between rainfall and disease cases. Despite this finding, the model required 99 instances before learning the mechanism, which does not pair well with data scarcity.

The other models, which interestingly performed well in the previous *MIMES*, did not learn the lagged relationship between rainfall and disease cases, which is not ideal as well. This exhibits their lack of ability to capture a well known DGP of VBD.

Another approach to evaluating this mechanism was to simulate random values between 0 and 50 for rainfall at each time step, opposed to simulating rainfall spikes as a fixed magnitude at random time steps. This was pursued to investigate if it would give other findings, but did in fact reflect the initial results from this *MIMES*.

Chapter 7

Discussion

This chapter discusses the outcome of the thesis, focusing on the framework, theoretical and practical contributions, as well as limitations and directions for future work.

7.1 Data Simulation

The iterative design process, combined with continuous evaluation of the framework, have contributed to a framework of high quality. This is especially the case for the DSL, and by extension the `Simulator` component. As highlighted in chapter 5, several iterations with improvements of the DSL was conducted. Section 2.7 points out that when developing a DSL, the underlying framework naturally occurs, which has also been the case for `mestDS`. In other words, changes to the DSL have resulted in changes to the `Simulator` component.

Following the guidelines of DSR have supported the development of a framework of high quality. In particular, the data simulation process demonstrates the following strengths:

- **Flexibility:** A primary focus during the development process, was to increase the flexibility of the framework, giving the user more control of the simulations. The framework started initially with predefined functions, which was parameterized later, given the user some flexibility. As the understanding of model evaluation and the DGP of VBD, the limitations of predefined functions became apparent. The final version requires the user to define all features of the simulation themselves, giving the user full control, and facilitates the simulation of essentially any possible time series. Additionally, it could be argued that by giving the user full control over the simulations, demands the user to define simulation configurations with intent, opposed to assigning random values to the parameters of a predefined function. Consequently, the resulting time series are likely to provide more meaningful insights during model evaluation.
- **Efficiency and re-usability:** As argued in section 2.5, a model should be evaluated on multiple datasets, to ensure a rigorous model evaluation. This demands the DSL to facilitate efficient and clear configurations of multiple simulations. In particular, the implementation of *inheritance* and a *public* space for functions and lists in the DSL, have significantly improved this aspect. The *MIMES* in chapter 6 demonstrated this strength, where multiple simulations, with minor differences, was required.

- **Abstraction and communication:** Fowler highlights the level abstraction, by decoupling domain logic from the complex underlying framework, that DSL adds, which also applies for this case [7]. With a brief understanding of how the DSL works, an external person could easily understand what the simulation configurations mean, by allowing them to ignore the complex underlying framework.

The framework, in regards to the DSL and data simulation, also pose some limitations. Firstly, giving the user full control over the simulation configurations, increases complexity and required work to the data simulation-pipeline. In addition, it increases the threshold for learning how to use the framework. Furthermore, demanding the user to define features themselves, requires prior Python and data simulation knowledge from the user that writes the DSL and an external person that reads it. This also impairs error handling, as writing Python functions in YAML-format, requires strict syntax for both DSL and Python code, without linting.

During the experiments, it became apparent that some functions are reused across multiple experiments, i.e. several DSL scripts included the same function. This requires a commonly used function to be defined for each script, and impedes efficiency and re-usability.

7.2 Model Evaluation

Many of the strengths present in the simulation process are also evident in the model evaluation process. Similar to the `Simulator`, the `Evaluator` component has benefited from the iterative development process of the DSL, where improvements naturally resulted in corresponding developments of the underlying framework:

- **Flexibility:** As previously discussed, flexibility has been a primary focus of the framework, and this applies to the model evaluation pipeline as well. Since chap-models require varying time deltas and prediction lengths, accommodating this diversity is essential for ensuring the framework's usability.

  The DSL further enhances flexibility by allowing the user to define the number of test sets, their length, and their stride, thereby providing precise control over which data is used for evaluation. For example, in the *MIMES* experiments, it was crucial that models were evaluated on time series including sudden outbreaks in order to validate whether the models captured the target of the experiments.

  Flexibility is also present in the choice of loss metrics. Since appropriate metrics vary depending on the characteristics of the data, the DSL enables the user to select multiple loss functions from a predefined function pool. Finally, the framework supports customization of the generated evaluation reports: users may specify the plotting range, which becomes especially valuable when long simulations make detailed inspection of plots difficult.

- **Abstraction and efficiency:** The level of abstraction that DSL provides, as discussed in the previous section, also applies for the model evaluation. The DSL allows the user to define a model evaluation, without worrying about the complex underlying framework. The simplistic and abstracted model evaluation configuration also enhances efficiency.

- **Loose coupling:** The introduction of loose coupling between components has substantially increased the framework's versatility. Where earlier versions restricted the `Evaluator` to function only with `Simulator` instances, the current design supports evaluation on any existing dataset, such as real-world data or previously simulated data.

Naturally, some limitations of the current model evaluation process remain. The literature highlights evaluation methods such as time series cross-validation and bootstrapping [18]. In contrast, the framework implements a simplified variant of holdout validation combined with time series cross-validation, where the model is trained once and then evaluated across multiple test sets. This approach is adequate when working with simulated data, where dataset size is theoretically unconstrained. However, when evaluating on real-world data, which is often scarce, more data-efficient evaluation methods, such as those recommended in the literature, would be preferable.

7.3 Theoretical Contribution

Throughout the thesis, valuable insights into data simulation, model evaluation, their intersection, and the development of a generic, flexible, and efficient framework have been gained:

- **DSL:** The thesis demonstrates that a DSL constitutes a viable solution for efficiently simulating multiple time series. The DSL has been crucial to the quality of the framework, and its benefits align with those highlighted by Fowler [7], such as improved efficiency, a level of abstraction, and clearer communication. An additional strength of DSLs, not directly emphasized by Fowler but evident in this thesis, is DSL's ability structure code and data. In particular, the *inheritance* key in `mestDS`'s DSL enables the reusability of existing configurations, further enhancing efficiency. This highlights how DSLs also can serve as a structural mechanisms, along with the benefits pointed out by Fowler.

- **MIMES:** Sandve et al. argue that simulated data should be given equal importance to real-world data in model evaluation [27]. They further suggest that models should first be tested on simplistic datasets before being applied to complex, real-world data. The use of data simulation as a tool for evaluating vector-borne disease forecasting models has received little attention in the literature. This thesis addresses this gap by exploring *MIMES* with forecasting models. Beyond this exploration, the thesis also contributes with practical guidance on how *MIMES* can be applied to the evaluation of vector-borne disease forecasting models, such as what DGP-mechanisms to explore.

7.4 Practical Contribution

In addition to the theoretical contributions discussed in the previous section, the `mestDS` framework provides practical value. The framework enables developers and evaluators of `chap-models` to simulate time series and evaluate models. It can be utilized as a holistic pipeline, from data simulation to model evaluation, or used separately due to the loose coupling of its components.

The generated reports allow external or domain experts to assess model performance, enabling interdisciplinary collaboration. Furthermore, by facilitating evaluation on

simulated time series with controlled signals, the framework can enhance model transparency and trustworthiness, as well as guide model development.

7.5 Future work

Future work should address limitations and continue the exploration of the *MIMES* approach.

First, the challenges related to the complexity of the DSL should be addressed. To overcome this challenge, the co-student that contributed to the development of the framework pursued the development of a framework that addresses this challenge. A framework that provides a pool with commonly used simulation functions was developed.

First, the complexity of the DSL increases the threshold for new users. While the current design provides flexibility and control, it also requires prior programming and simulation expertise. To lower this threshold, future development could explore solutions that balance flexibility with usability. For instance, the co-student that contributed to the development of the framework has already pursued work on the framework with a pool of commonly used simulation functions.

Second, the chap-models used during the experiments performed poorly on simplistic simulations, where time lag between rainfall and disease cases was introduced. Additionally, the models primarily included climatic variables. While the results highlighted model weaknesses (and the strength of *MIMES*), they also prohibited the exploration of more complex applications of *MIMES*. Addressing the challenges related to time lag and adding demographic, entomological, or climate change-related variables should improve model performance [18], and allow for further exploration of *MIMES*. For instance, by adding information such as whether or not a region has piped water, which affects the prevalence of VBD and consequently is a DGP, simplistic *MIMES* regarding this mechanism could be explored. Furthermore, the experiments conducted only included one region in the dataset, while some existing datasets include multiple regions. Future work should include the exploration of *MIMES* with multiple regions included in the dataset.

Third, using real-world data on models with the framework has not been prioritized during the thesis, but is a direction for future work. Comparing outcomes from *MIMES* and real-world data evaluations, could provide valuable insights into the value of the *MIMES* approach. For example, results from an evaluation on real-world dataset that includes clear seasonal cycles could be compared to the *MIMES* discussed in section 6.1.1. However, as data is scarce, finding real-world datasets that exhibit targeted mechanisms could be challenging. Closely related to this is the potential implementation of alternative evaluation methods. Methods more suited for small real-world datasets could be implemented to the framework, increasing the framework's usability in practical settings.

Lastly, the results from the experiments are *technically* based on only one evaluation for each time series simulated, per model. The framework should be extended to report a result from multiple iterations of model evaluation on the same time series, i.e. a given model should be trained and evaluated multiple times on a time series. This would increase the trustworthiness of the reported result and the confidence that a model has actually learns the pattern, and rule out the chance of luck. The evaluations that was conducted during the experiments was indeed conducted multiple times for these reasons.

Chapter 8

Conclusion

The thesis' objective was to develop a generic and efficient *data simulation* framework for evaluation of VBD forecasting models. The thesis also explores how simulated time series can enhance model evaluation, transparency and trustworthiness. In particular, the thesis sought to contribute with a practical artifact for and new insights into the intersection of data simulation and model evaluation, for VBD forecasting.

The research followed a DSR approach, which ensured research rigor and a quality artifact. This process resulted in the framework `mestDS`, which facilitates flexible and efficient simulation and model evaluation. A key feature of the framework is its DSL, which also contributes with a level of abstraction by decoupling the domain logic and the complex underlying framework.

The latter part of the thesis explored the potential of a complementary approach to model evaluation, referred to as *MIMES*. This approach utilized the framework to evaluate models on simplistic time series simulations that targeted specific mechanisms of VBD's DGP. The experiments highlighted *MIMES*' valuable insights into a models performance of various mechanisms of the DGP and their inherent learning requirements. Furthermore, the experiments demonstrated that the framework truly provides a flexible and efficient pipeline from simulation to model insights.

The research's contribution also revealed some limitations. As flexibility in the framework increased, so did its complexity, and consequently the learning threshold for new users. The experiments demonstrated that the evaluated models performed poorly on simplistic tests. In addition, the models mainly consisted of climatic variables, contradictory to literature's recommendations. These factors impeded the exploration of more complex applications of *MIMES*. Lastly, the evaluation on real-world data was not prioritized, even though the framework supports it.

Future work should address these limitations, particularly by incorporating non-climatic variables relevant to VBD dynamics and improve model performance on mechanisms of the DGP, followed by further exploration of *MIMES*, and the extension to evaluations on real-world datasets. Comparing insights from *MIMES* and real-world evaluations could provide a deeper understanding of the value of the *MIMES* approach.

To conclude, the DSR paradigm has ensured a framework of high quality, providing flexibility, efficiency and abstraction to data simulation and model evaluation. The thesis has demonstrated that simulated time series can play a meaningful role in enhancing the evaluation of VBD forecasting models, providing both transparency and trustworthiness. The thesis contributes with a practical tool, `mestDS`, for researchers and practitioners developing and evaluating `chap-models`, and theoretical insights into the use of DSL and *MIMES*. These contributions lay the foundation for future work on improving model

evaluation on simulated time series, and potentially improve disease forecasting models performance and reliability.

Bibliography

[1] Ioana Agache et al. "Climate change and global health: A call to more research and more action". In: *Allergy* 77.5 (2022). _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/all.15229, pp. 1389–1407. ISSN: 1398-9995. DOI: 10.1111/all.15229. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/all.15229 (visited on 05/02/2025).

[2] Josu Arteche. "Bootstrapping long memory time series: Application in low frequency estimators". In: *Econometrics and Statistics* 29 (Jan. 2024). Publisher: Elsevier BV, pp. 1–15. ISSN: 2452-3062. DOI: 10.1016/j.ecosta.2021.06.002. URL: https://linkinghub.elsevier.com/retrieve/pii/S2452306221000769 (visited on 28/07/2025).

[3] *Climate change*. URL: https://www.who.int/news-room/fact-sheets/detail/climate-change-and-health (visited on 19/10/2025).

[4] *DHIS2 & HISP*. DHIS2. URL: https://dhis2.org/dhis2-and-hisp/ (visited on 08/11/2025).

[5] *DHIS2 Modeling*. URL: https://chap.dhis2.org/ (visited on 08/11/2025).

[6] Isabel K. Fletcher et al. "Climate services for health: From global observations to local interventions". In: *Med* 2.4 (9th Apr. 2021), pp. 355–361. ISSN: 2666-6340. DOI: 10.1016/j.medj.2021.03.010. URL: https://www.sciencedirect.com/science/article/pii/S2666634021001124 (visited on 18/02/2025).

[7] Martin Fowler. *Chapter 2. Using Domain-Specific Languages*. ISBN: 978-0-13-210754-9. URL: https://learning.oreilly.com/library/view/domain-specific-languages/9780132107549/ch02.html (visited on 14/07/2025).

[8] Fanzhe Fu et al. *Are Synthetic Time-series Data Really not as Good as Real Data?* 1st Feb. 2024. DOI: 10.48550/arXiv.2402.00607. arXiv: 2402.00607[cs]. URL: http://arxiv.org/abs/2402.00607 (visited on 28/07/2025).

[9] Kenneth L. Gage et al. "Climate and Vectorborne Diseases". In: *American Journal of Preventive Medicine* 35.5 (Nov. 2008), pp. 436–450. ISSN: 07493797. DOI: 10.1016/j.amepre.2008.08.030. URL: https://linkinghub.elsevier.com/retrieve/pii/S074937970800706X (visited on 11/02/2025).

[10] Alan R. Hevner et al. "Design Science in Information Systems Research". In: *MIS Quarterly* 28.1 (2004), pp. 75–105. DOI: 10.2307/25148625.

[11] *HISP History*. DHIS2. URL: https://dhis2.org/hisp-history/ (visited on 08/11/2025).

[12] *HISP Network*. DHIS2. URL: https://dhis2.org/hisp-network/ (visited on 08/11/2025).

[13] *Information Systems (IS) - Department of Informatics*. URL: https://www.mn.uio.no/ifi/english/research/groups/is/index.html (visited on 08/11/2025).

[14] Vector Disease Control international. *Larval Mosquito Habitats*. Vector Disease Control International. 11th Aug. 2016. URL: https://www.vdci.net/blog/larval-mosquito-habitats/ (visited on 19/02/2025).

[15] Micheal A. Johansson et al. *An open challenge to advance probabilistic forecasting for dengue epidemics*. DOI: 10.1073/pnas.1909865116. URL: https://www.pnas.org/doi/epdf/10.1073/pnas.1909865116 (visited on 11/02/2025).

[16] Manu Joseph and Jeffrey Tackes. "Modern Time Series Forecasting with Python: Second Edition". In: Packt / O'Reilly, 2024. Chap. 1. ISBN: 978-1-83588-318-1.

[17] Diederik P. Kingma and Max Welling. "An Introduction to Variational Autoencoders". In: *Foundations and Trends® in Machine Learning* 12.4 (2019), pp. 307–392. ISSN: 1935-8237, 1935-8245. DOI: 10.1561/2200000056. arXiv: 1906.02691[cs]. URL: http://arxiv.org/abs/1906.02691 (visited on 31/07/2025).

[18] Xing Yu Leung et al. "A systematic review of dengue outbreak prediction models: Current scenario and future directions". In: *PLOS Neglected Tropical Diseases* 17.2 (13th Feb. 2023), e0010631. ISSN: 1935-2727. DOI: 10.1371/journal.pntd.0010631. URL: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9956653/ (visited on 04/06/2025).

[19] *Machine Learning Model Evaluation*. GeeksforGeeks. Section: Technical Scripter. URL: https://www.geeksforgeeks.org/machine-learning/machine-learning-model-evaluation/ (visited on 08/08/2025).

[20] Serge Mankovski. "Loose Coupling". In: *Encyclopedia of Database Systems*. Springer, New York, NY, 2018, pp. 2155–2156. ISBN: 978-1-4614-8265-9. DOI: 10.1007/978-1-4614-8265-9_1187. URL: https://link.springer.com/rwe/10.1007/978-1-4614-8265-9_1187 (visited on 12/07/2025).

[21] *MLflow Projects | MLflow*. URL: https://mlflow.org/docs/latest/ml/projects/ (visited on 08/11/2025).

[22] Karel G M Moons et al. "Risk prediction models: II. External validation, model updating, and impact assessment". In: *Heart* 98.9 (1st May 2012). Publisher: BMJ, pp. 691–698. ISSN: 1355-6037, 1468-201X. DOI: 10.1136/heartjnl-2011-301247. URL: https://heart.bmj.com/lookup/doi/10.1136/heartjnl-2011-301247 (visited on 06/08/2025).

[23] Sebastian Raschka. *Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning*. 11th Nov. 2020. DOI: 10.48550/arXiv.1811.12808. arXiv: 1811.12808[cs]. URL: http://arxiv.org/abs/1811.12808 (visited on 08/08/2025).

[24] Filipe Rocha et al. "Understanding dengue fever dynamics: a study of seasonality in vector-borne disease models". In: *International Journal of Computer Mathematics* 93.8 (2nd Aug. 2016), pp. 1405–1422. ISSN: 0020-7160, 1029-0265. DOI: 10.1080/00207160.2015.1050961. URL: http://www.tandfonline.com/doi/full/10.1080/00207160.2015.1050961 (visited on 18/08/2025).

[25] Joacim Rocklöv and Robert Dubrow. "Climate change: an enduring challenge for vector-borne disease prevention and control". In: *Nature Immunology* 21.5 (May 2020). Publisher: Nature Publishing Group, pp. 479–483. ISSN: 1529-2916. DOI: 10.1038/s41590-020-0648-y. URL: https://www.nature.com/articles/s41590-020-0648-y (visited on 18/02/2025).

[26] Luis Roque et al. *Cherry-Picking in Time Series Forecasting: How to Select Datasets to Make Your Model Shine*. 19th Dec. 2024. DOI: 10.48550/arXiv.2412.14435. arXiv: 2412.14435[cs]. URL: http://arxiv.org/abs/2412.14435 (visited on 13/08/2025).

[27] Geir Kjetil Sandve and Victor Greiff. "Access to ground truth at unconstrained size makes simulated data as indispensable as experimental data for bioinformatics methods development and benchmarking". In: *Bioinformatics (Oxford, England)* 38.21 (31st Oct. 2022), pp. 4994–4996. ISSN: 1367-4811. DOI: 10.1093/bioinformatics/btac612.

[28] Hemanshu Singh and Maya Rajnarayan Ray. "Synthetic Stream Flow Generation of River Gomti Using ARIMA Model". In: *Advances in Civil Engineering and Infrastructural Development*. Ed. by Laxmikant Madanmanohar Gupta et al. Singapore: Springer Singapore, 2021, pp. 255–263. ISBN: 978-981-15-6463-5.

[29] George C.M. Siontis et al. "External validation of new risk prediction models is infrequent and reveals worse prognostic discrimination". In: *Journal of Clinical Epidemiology* 68.1 (Jan. 2015). Publisher: Elsevier BV, pp. 25–34. ISSN: 0895-4356. DOI: 10.1016/j.jclinepi.2014.09.007. URL: https://linkinghub.elsevier.com/retrieve/pii/S0895435614003539 (visited on 06/08/2025).

[30] Adam B. Smith. *U.S. Billion-dollar Weather and Climate Disasters, 1980 - present (NCEI Accession 0209268)*. 2020. DOI: 10.25921/STKW-7W73. URL: https://www.ncei.noaa.gov/archive/accession/0209268 (visited on 05/02/2025).

[31] *Vector-borne diseases*. URL: https://www.who.int/news-room/fact-sheets/detail/vector-borne-diseases (visited on 24/07/2025).

[32] *What is Loose Coupling? | Definition from TechTarget*. Search Networking. URL: https://www.techtarget.com/searchnetworking/definition/loose-coupling (visited on 12/07/2025).

[33] *WMO confirms 2024 as warmest year on record at about 1.55°C above pre-industrial level*. World Meteorological Organization. 10th Jan. 2025. URL: https://wmo.int/news/media-centre/wmo-confirms-2024-warmest-year-record-about-155degc-above-pre-industrial-level (visited on 05/02/2025).

[34] World Health Organization and UNICEF/UNDP/World Bank/WHO Special Programme for Research and Training in Tropical Diseases. *Global vector control response 2017-2030*. Geneva: World Health Organization, 2017. 51 pp. ISBN: 978-92-4-151297-8. URL: https://iris.who.int/handle/10665/259205 (visited on 24/07/2025).

[35] World Meteorological Organization. *State of the Global Climate 2023*. Erscheinungsort nicht ermittelbar: United Nations, 2024. 1 p. ISBN: 978-92-63-11347-4.

[36] Jinsung Yoon et al. "Time-series Generative Adversarial Networks". In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper_files/paper/2019/hash/c9efe5f26cd17ba6216bbe2a7d26d490-Abstract.html (visited on 31/07/2025).

Appendix A

mestDS Links

- Source code: https://github.com/martin-og-ingar/mestDS/tree/ingar/pipeline-model-assesment
- Installation guide and tutorial: https://github.com/martin-og-ingar/mestDS-tutorial
- PyPi: https://pypi.org/project/mestDS/

Appendix B

Example DSL Configuration

```
public:
  functions:
    get_flat_value: |
      def get_flat_value(mean, randomness):
        return np.random.normal(mean, randomness)
    get_rainfall_spike: |
      def get_rainfall(i, seasonal_spikes, random_spikes):
        if i in seasonal_spikes or i in random_spikes:
          return 50
        else:
          return 10
    get_rainfall_spike_2: |
      def get_rainfall(i, random_spikes):
        if i in random_spikes:
          return 50
        else:
          return 10
    get_disease_cases_with_lag: |
      def get_disease_cases_with_lag(rainfall, lag):
        if len(rainfall) < lag:
          return int(rainfall[-1])
        return int(rainfall[-lag])
  lists:
    seasonal_spikes: [5, 17, 29, 41, 53, 65, 77, 89]

simulators:
  - id: 1
    name: train with no random spike
    description: Is the model detecting the seasonality or time lag? The model i
    time_delta: M
    length: 100
    x:
      - name: rainfall
        function_ref: get_rainfall_spike
        params:
          random_spikes: [93]
```

```
      - name: population
        function_ref: get_flat_value
        params:
          mean: 1000
          randomness: 5
      - name: mean_temperature
        function_ref: get_flat_value
        params:
          mean: 27
          randomness: 3
    y:
      - name: disease_cases
        function_ref: get_disease_cases_with_lag
        params:
          lag: 3
  - id: 2
    inherit: 1
    name: train with one random spike
    description: In this case, the model is trained with one random spike a
    x:
      - name: rainfall
        params:
          random_spikes: [56, 93]
  - id: 3
    inherit: 1
    name: train with two random spikes
    description: In this case, the model is trained with two random spikes
    x:
      - name: rainfall
        params:
          random_spikes: [23, 56, 93]
  - id: 4
    inherit: 1
    name: train with three random spikes
    description: In this case, the model is trained with three random spike
    x:
      - name: rainfall
        params:
          random_spikes: [23, 37, 64, 93]
  - id: 5
    inherit: 1
    name: train with four random spikes
    description: In this case, the model is trained with four random spikes
    x:
      - name: rainfall
        params:
          random_spikes: [23, 37, 56, 72, 93]
  - id: 6
    inherit: 1
```

```
    name: only random spikes
    description: In this case, the model is train and predicts on random spikes
    x:
      - name: rainfall
        function_ref: get_rainfall_spike_2
        params:
          random_spikes:
            [3, 7, 14, 19, 26, 35, 41, 49, 55, 59, 66, 77, 84, 90, 95]

evaluators:
  - model: https://github.com/dhis2-chap/ewars_template.git
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    metrics: [mse, pocid, theils_u]
    plot_length: 100
  - model: "https://github.com/sandvelab/chap_auto_ewars@58d56f86641f4c7b09bbb6
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    metrics: [mse, pocid, theils_u]
    plot_length: 40
  - model: https://github.com/sandvelab/monthly_ar_model@cadd785872624b4bcd839a
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    metrics: [mse, pocid, theils_u]
    plot_length: 40
```

Listing B.1: Full DSL example used to define simulations and evaluations.

Appendix C

MIMES 2 - DSL configuration

```
public:
  functions:
    get_flat_value: |
      def get_flat_value(mean, randomness):
        return np.random.normal(mean, randomness)
    get_rainfall_spike: |
      def get_rainfall(i, random_spikes):
        if i in random_spikes:
          return 50
        else:
          return 10
    get_rainfall_period: |
      def get_rainfall(i, random_spikes, period_length):
        for x in range(0, period_length):
          if (i-x) in random_spikes:
            return 50
        return 10
    get_disease_cases_with_lag: |
      def get_disease_cases_with_lag(rainfall, lag):
        if len(rainfall) < lag:
          return int(rainfall[-1])
        return int(rainfall[-lag])
  lists:
    random_spikes: [
        4,
        7,
        10,
        18,
        22,
        26,
        35,
        37,
        39,
        41,
        45,
        46,
```

```
52,
57,
58,
67,
70,
78,
81,
82,
86,
94,
95,
96,
100,
107,
115,
125,
130,
132,
137,
140,
146,
157,
158,
165,
171,
176,
184,
189,
193,
195,
206,
207,
208,
210,
211,
217,
218,
222,
225
231,
236,
251,
252,
254,
255,
263,
273,
277,
282,
```

```
        284,
        290,
        295,
        306,
        308,
        312,
        314,
        319,
        326,
        332,
        334,
        353,
        357,
        360,
        364,
        366,
        374,
        384,
        387,
        390,
        391,
        393,
        398,
        404,
        405,
        406,
        408,
        409,
        415,
        418,
        432,
        438,
        439,
        440,
        442,
        450,
        454,
        467,
        471,
        482,
        489,
        493,
      ]

simulators:
  - id: 1
    name: random spikes for 90 months (21 spikes)
    description: Is the model able to detect the lag relationship between rainfa
    time_delta: M
```

```
    length: 90
    x:
      - name: rainfall
        function_ref: get_rainfall_spike
      - name: population
        function_ref: get_flat_value
        params:
          mean: 1000
          randomness: 5
      - name: mean_temperature
        function_ref: get_flat_value
        params:
          mean: 27
          randomness: 0
    y:
      - name: disease_cases
        function_ref: get_disease_cases_with_lag
        params:
          lag: 2
  - id: 3
    inherit: 1
    name: random spikes for 120 months (25 spikes)
    length: 120
  - id: 4
    inherit: 1
    name: random spikes for 150 months (32 spikes)
    length: 150
  - id: 5
    inherit: 1
    name: random spikes for 180 months (38 spikes)
    length: 180
  - id: 6
    inherit: 1
    name: random spikes for 230 months (49 spikes)
    length: 230
  - id: 7
    inherit: 1
    name: random spikes for 270 months (55 spikes)
    length: 270
  - id: 8
    inherit: 1
    name: random spikes for 300 months (60 spikes)
    length: 300
  - id: 9
    inherit: 1
    name: random spikes for 330 months (67 spikes)
    length: 330
  - id: 10
    inherit: 1
```

```
    name: random spikes for 370 months (73 spikes)
    length: 370
  - id: 11
    inherit: 1
    name: random spikes for 420 months (87 spikes)
    length: 420
  - id: 12
    inherit: 1
    name: random spikes for 280 months (99 spikes)
    length: 500
evaluators:
  - model: https://github.com/dhis2-chap/ewars_template.git
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    metrics: [mse, pocid, theils_u]
  - model: "https://github.com/sandvelab/chap_auto_ewars"
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    metrics: [mse, pocid, theils_u]
  - model: https://github.com/sandvelab/monthly_ar_model
    prediction_length: 3
    stride: 2
    n_test_sets: 6
    metrics: [mse, pocid, theils_u]
```

Listing C.1: DSL configuration for "MIMES 2